

순서가 중요하다

오늘 풀 문제

정의부터 시작하지 않음

오늘의 흐름

- 문제를 만남
- 지금 아는 방법으로 풀어봄
- 어디서 막히는지 봄
- 막힌 것을 푸는 방법을 배움

이 흐름이 세 번 반복됨

9 3 1 0 7 5 8 6 8 4

3 다음에 7이 나왔는가

9 3 1 0 7 5 8 6 8 4

- 3은 두 번째, 7은 다섯 번째임
- 3이 먼저 나왔음

답 — 네

방금 여러분이 한 것

- 왼쪽부터 오른쪽으로 하나씩 읽음
 - 3을 본 순간부터 그 사실을 들고 끝까지 감
- 이 두 가지가 오늘 계속 나옴

9 7 1 0 3 5 8 6 8 4

- 7이 두 번째, 3이 다섯 번째임
- 7이 먼저 나왔음
- 3도 7도 둘 다 들어 있음
- 묻는 것은 있느냐가 아니라 어느 쪽이 먼저냐임

답 — 아니요

9 3 1 0 7 5 8 6 8 4

9 7 1 0 3 5 8 6 8 4

- 들어 있는 숫자가 그대로 같음 — 8이 두 번인 것까지
- 합도 평균도 가장 큰 수도 같음
- 다른 것은 순서뿐임
- 합·평균·최댓값·개수는 순서를 보지 않는 계산임

순서가 뒤집히면 의미가 뒤집힘

장바구니에 담음 → 결제함

결제함 → 장바구니에 담음

개가 사람을 물었다

사람이 개를 물었다

- 재료는 같음 · 벌어진 일은 다름

가장 단순한 방법

- 숫자 열 개를 전부 더함
- 합 하나를 입력으로 넣고 분류함
- 지금까지 만들어 온 분류기와 같은 구조임

한 줄 = 0~9 숫자 열 개 · 3 과 7 은 하나씩 · 학습 6,000줄 · 평가 2,000줄

정확도가 몇 % 일 것 같은가

채팅에 숫자만 적을 것

예측 ①

▶ 합 하나로 풀면 몇 % 인가 · 답 보기
실행 결과 · 노트북 L0 셀

50%는 아무것도 배우지 못했다는 뜻임

- 합 하나에는 3 과 7 중 무엇이 먼저였는지가 남아 있지 않음
- 데이터 · 층 · 학습 횟수를 늘려도 없는 정보는 생기지 않음
- 모델이 작아서 생긴 문제가 아님
- 입력에 답이 들어 있지 않아서 생긴 문제임

9 3 1 0 7 5 8 6 8 4

합 51

9 7 1 0 3 5 8 6 8 4

합 51

합을 계산하는 순간 순서를 버린 것임

51 하나만 보고 원래 줄을 되돌릴 방법은 없음

합이 나쁜 방법인 것은 아님

- 질문 A — 3 다음에 7이 나왔는가 → 48.8%
- 질문 B — 합이 45 이상인가 → 96%
- 같은 데이터임 · 질문만 바뀌었음

실행 결과 · 노트북 '질문을 바꾸면 — 합이 45 이상인가' 셀

오늘 가져갈 첫 번째 문장

순서를 섞어도 답이 같은가

순서가 중요한지는 데이터가 아니라 질문이 정함

새 데이터를 만나면 먼저 할 것

- 한 줄 안에서 칸의 순서를 섞어서 돌려봄
- 성능이 그대로임 → 순서는 이 문제에서 중요하지 않음
- 성능이 뚝 떨어짐 → 순서가 답을 정하고 있음

줄끼리 섞는 것이 아님

사용자 A



사용자 B



사용자 C

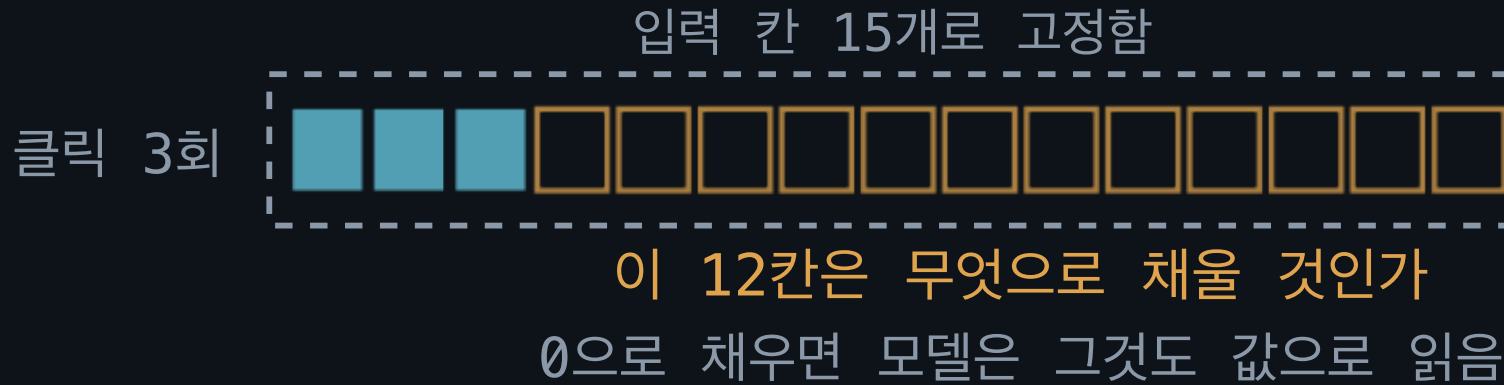


사용자 D



끝이 제각각임

길이가 매번 다름



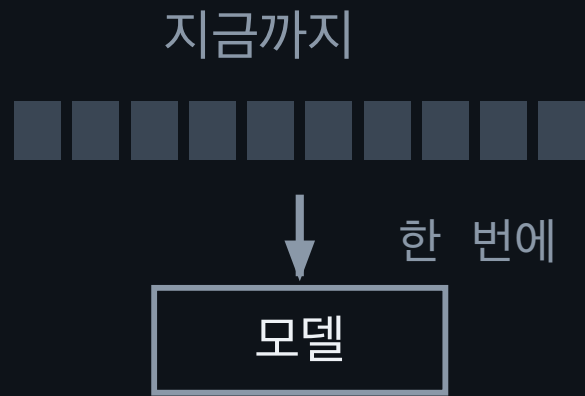
15개보다 긴 데이터가 오면
어떻게 할 것인가

순서를 섞어도 답이 같은가
다르다면, 길이도 매번 다른가

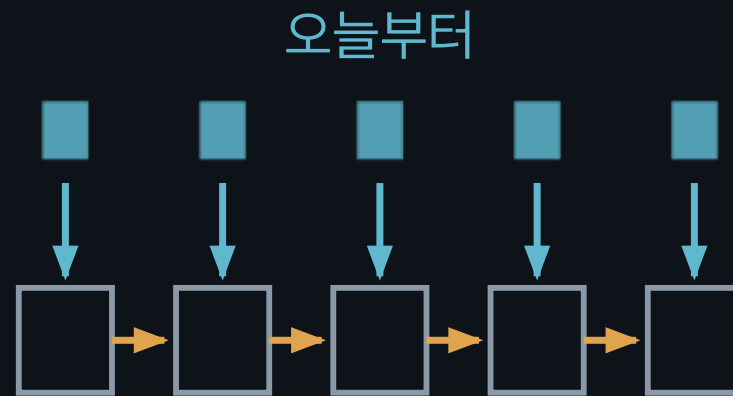
두 번째 갈래는 점심 전에 다시 꺼냄

순서를 안 버리는 방법

- 숫자를 하나씩 순서대로 읽음
- 첫 번째를 보고, 두 번째를 보고, 세 번째를 봄
- 눈으로 세어볼 때 했던 것과 같음

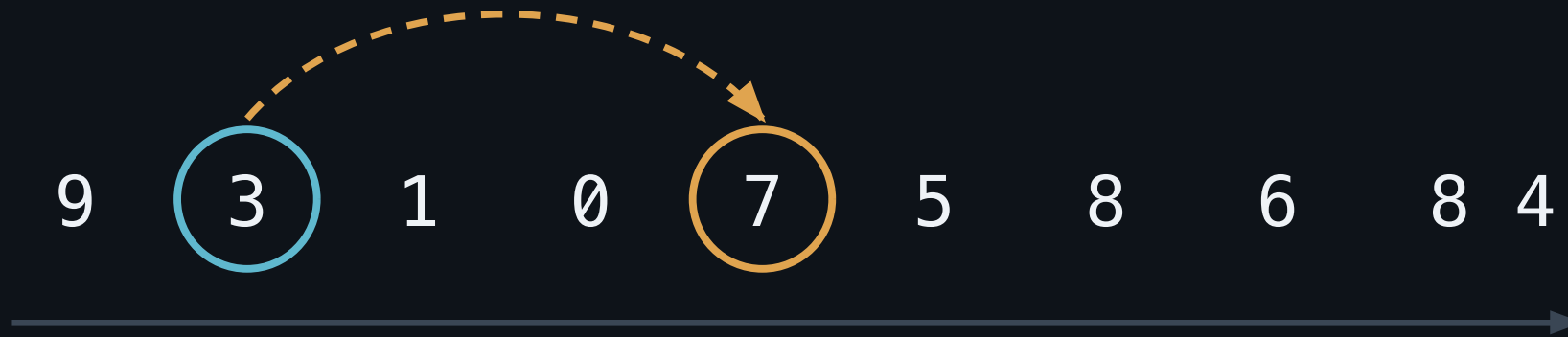


입력이 한 덩어리임
순서가 사라짐



순서가 남음

3이 나왔다는 것을 기억하고 있어야 함



앞에서 뒤로 무언가를 넘겨야 함

무엇을 넘길 것인가

- 전부 넘길 것인가 — 뒤로 갈수록 짐이 무거워짐
- 마지막 하나만 넘길 것인가 — 그 전 것은 잊힘
- 둘 다 아닌 다른 무언가를 넘길 것인가

무엇을 넘길 것인가

아까 남겨둔 질문

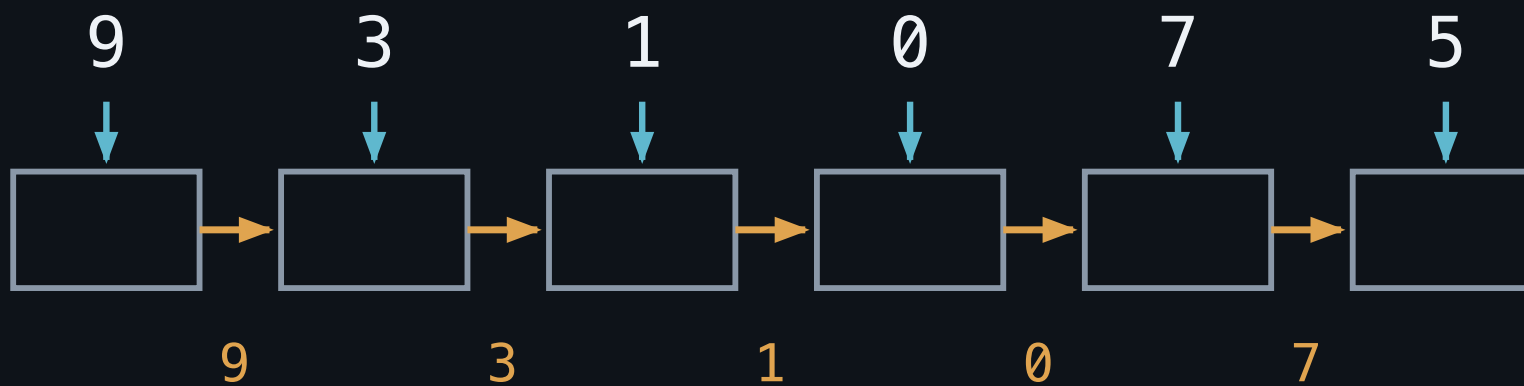
- 하나씩 순서대로 읽어야 함
- 앞에서 뒤로 무언가를 넘겨야 함

무엇을 넘길 것인가

첫 번째 후보

마지막 값 하나만 넘김

- 직전에 본 숫자 하나만 다음 칸으로 넘김
- 가장 싸고 가장 단순함



넘어가는 것은 직전 숫자 하나뿐임

이 방법은 몇 %일까

찍기는 50% 임 · 합으로 푼 것은 48.8% 였음

예측 ②

▶ 마지막 값 하나로 풀면 몇 % 인가 · 답 보기

실행 결과 · 노트북 '나머지 세 가지 — 평균 · 마지막 값 · 첫 값' 셀

방금 나온 세 줄

- 합은 아까 48.8% 였음
- 평균 · 마지막 값 · 첫 값을 넘긴 결과가 찍혔음
- 숫자마다 왜 그만큼인지 생각해 볼 것

합이 50 근처인 이유는 아까 봤음

나온 결과

- 마지막 값 — 60.4%
- 첫 값 — 60.9%
- 합 — 48.8%
- 평균 — 49.9%

어느 것도 60% 근처를 크게 못 넘음

왜 하필 60% 인가

▶ 마지막 칸이 7이면 답은 무엇인가 · 답 보기

계산해 보면

- 열 칸 중 3 과 7 이 차지하는 칸은 두 개임
- 마지막 칸이 그 두 칸 중 하나 — 10 분의 2 · 확실히 맞힘
- 나머지 10 분의 8 — 단서 없음 · 찍기

$$0.2 \times 1 + 0.8 \times 0.5 = 0.6$$

노트북 '마지막 칸 세어보기' 셀로 실제로 셈

첫 번째 후보의 판정

- 넘긴 것이 하나뿐이라 대부분의 경우 단서가 없음
- 표식이 마지막 칸에 온 경우만 맞힘 · 첫 값은 첫 칸
- 문제는 모델이 아니라 무엇을 넘겼는지에 있음

모델을 키워도 60% 근처 · 가장 큰 모델은 48.8% · 노트북 '모델을 키우면' 셀 · 미리 돌린 결과

네 가지의 공통점

▶ 네 가지의 공통점은 무엇인가 · 답 보기

그럼 더 많이 넘기면 되지 않을까

다음 후보로 넘어감

두 번째 후보 — 전부 넘김

- 하나만 넘겨서 부족했으니 다 넘김
- 정보를 하나도 안 버림

하는 방법이 두 가지임 · 둘 다 해봄

먼저 — 통째로 펼쳐서 한 번에

- 길이 10 → 100%
- 길이 20 → 99.6%
- 길이 50 → 95.9%
- 길이 100 → 88.4%

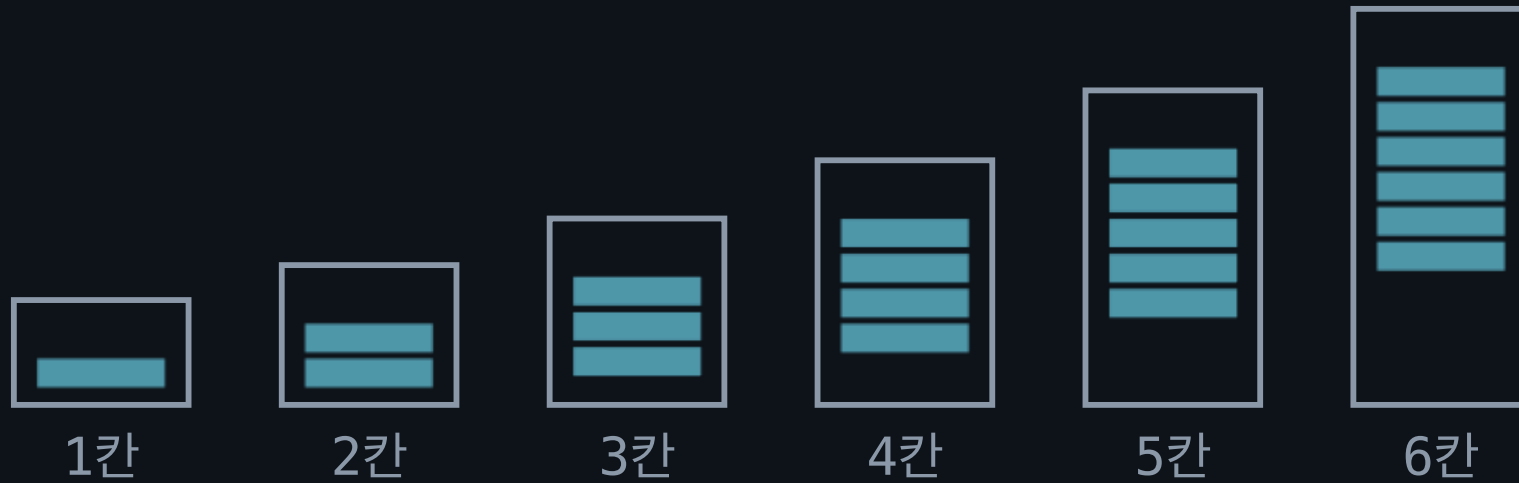
실행 결과 · 노트북 '통째로 펼쳐서 한 번에 넣으면' 셀

길이 50 에서 데이터를 줄여보면

- 학습 데이터 6,000건 → 95.9%
- 2,000건 → 89.4%
- 500건 → 67.0%

파라미터 17,154개 · 데이터가 모자람

실행 결과 · 노트북 '데이터 양을 줄이면' 셀



뒤로 갈수록 넘기는 짐이 늘어남

크기가 다르면 모델도 달라짐

- 3번째 칸 → 입력 2개짜리 모델이 필요함
- 10번째 칸 → 입력 9개짜리 모델이 필요함
- 두 모델은 가중치를 같이 쓸 수 없음
- 200칸이면 모델이 200개임

왜 학습이 안 되는가

- 데이터가 1,000건이면 200번째 모델도 1,000번만 봄
- 그런데 200번째 모델은 파라미터가 훨씬 많음
- 배울 것은 많은데 볼 기회는 같음

아까 길이 50에서 본 것과 같은 일임

그리고 아침의 그 질문

- 길이가 매번 다르면 가장 긴 것에 맞춰야 함
- 그보다 긴 데이터가 오면 방법이 없음

순서 문제를 길이 문제로 바꿔놓았을 뿐임

그럼 실제로 무엇이 필요한가

시험 삼아 두 가지만 넘겨봄

- 3 을 봤는가 · 7 을 봤는가

51.2%

필요한 것은 '봤는가' 가 아님

- 3 도 7 도 언제나 둘 다 들어 있음 → 두 값이 늘 같음
 - 필요한 것은 어느 쪽을 먼저 봤는가 임
 - 아직 둘 다 안 봄 · 3 이 먼저 · 7 이 먼저
- 상태 세 개로 바꾸면 → 100%

지금까지 우리가 한 것
무엇을 넘길지
사람이 정했음

우리가 원래 하던 방식

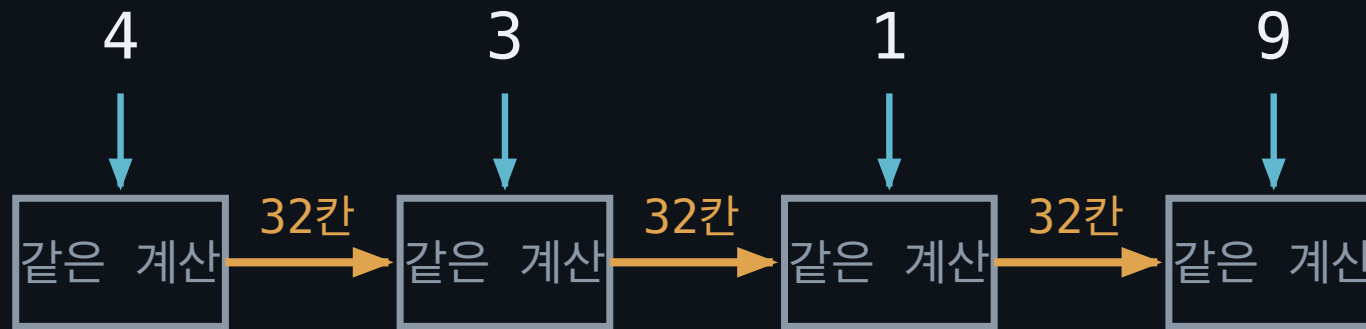
- 이미지에서 무엇을 볼지 사람이 정하지 않음
- 어떤 특징이 중요한지를 모델이 학습함

넘길 것도 학습시키면 되지 않을까

학습시키려면 조건이 둘 있음

- 넘기는 것의 크기가 고정이어야 함
- 매 칸에서 갱신되어야 함

크기가 고정이라야 같은 모델을 계속 쓸 수 있음



넘어가는 요약은 항상 같은 크기임

그 안에 무엇을 담을지는 학습으로 정해짐

이 요약에 붙은 이름

hidden state

숨은 상태 · 은닉 상태

헛갈리기 쉬운 지점

- 지금까지 본 것이 차곡차곡 쌓이는 창고가 아님
- 매 칸에서 새로 쓰이는 요약 임
- 새 요약에는 이전 요약의 흔적이 남아 있음

한 줄로 쓰면

새 요약 = 계산(이전 요약 , 새 숫자)

기호로 쓰면

$$h_t = f(h_{t-1} , x_t)$$

오늘 가져갈 두 번째 문장

순서가 중요하다면 무엇을 넘길 것인가

사람이 정할까 · 학습시킬까

아직 안 푼 문제가 하나 있음

- 칸마다 모델이 달라지는 문제
- 200칸이면 모델 200개였음
- 데이터는 같은데 배울 것만 늘어났음

넘기는 것의 크기가 고정임

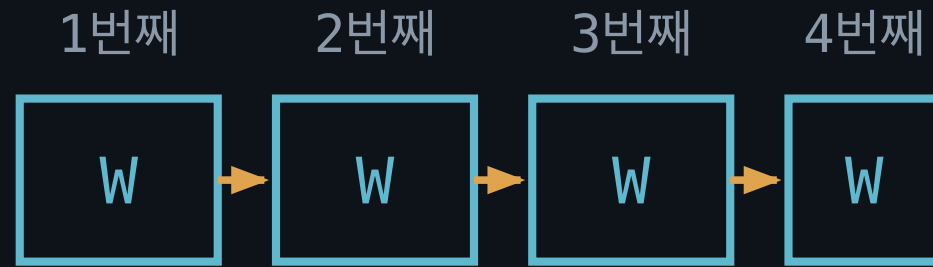
- 1번째 칸도 32칸을 받음
- 200번째 칸도 32칸을 받음
- 받는 크기가 같으면 같은 모델을 쓸 수 있음

실제로 있는 것



상자 하나
자기에게 돌아옴

그림으로 펼친 것



네 상자가 전부 같은 W 임
시간을 옆으로 늘어놓은 것뿐임

CNN - 공간을 훑음



같은 필터를
위치마다 씬

RNN - 시간을 훑음



같은 발상임

같은 가중치를
순간마다 씬

이 성질에 붙은 이름 파라미터 공유

매 순간이 같은 가중치를 나눠 씀

그래서 실제로 세어보면

- 길이 10 → 파라미터 1,474개
- 길이 50 → 1,474개
- 길이 200 → 1,474개

스텝이 20배가 되어도 그대로임

통째로 펼치기는 길이 50 에서 17,154개였음

실행 결과 · 노트북 L1 셀

길이 문제도 같이 풀림

- 모델이 길이를 모름 → 몇 번 돌든 상관없음
- 학습할 때 길이 10, 쓸 때 길이 200 도 가능함

아침에 심어둔 두 번째 질문의 절반이 풀린 것임

지금까지 세 가지

- 넘기는 것의 크기를 고정함
- 그 안에 무엇을 담을지는 학습시킴
- 매 순간 같은 계산을 씀

이 셋이 모이면 **RNN** 임

지금부터 할 것

- 방금 그림을 손으로 코드에 옮김
 - 그다음 PyTorch 가 준비해 둔 것과 맞춰 봄
- 먼저 손으로 짜는 이유가 있음

상자 하나 만들기

- 10 — 들어오는 숫자의 표현 크기
- 32 — 넘길 요약의 크기
- 이 한 줄이 상자 하나 임

for 문으로 돌리기

세 줄이 전부임

다 돌고 나면

- 4 — 한 번에 처리한 시퀀스 개수
- 32 — 요약 크기
- 길이 10 은 사라졌음

실행 결과 · 노트북 '손으로 펼친 판' 셀

방금 코드가 그림 그대로임

▶ 파라미터 공유는 코드의 어디에 보이는가 · 답 보기

PyTorch 가 이미 만들어 둔 것

- for 문이 사라졌음
- 안에서 같은 일을 하고 있음

손 코드와 맞춰보면

같은 가중치를 넣고 돌린 결과가 완전히 일치함
실행 결과 · 노트북 'nn.RNN 과 맞춰 보기' 셀

돌려주는 것이 두 개임

- output — 매 칸의 요약을 전부 모은 것
- h_n — 마지막 칸의 요약

지금은 둘이 사실상 같음

정말 똑같은지 · 언제 달라지는지는 조금 뒤에 봄

이제 학습시켜 봄

- 마지막 요약을 받아 '네/아니요' 를 답하게 함
- 나머지는 늘 하던 학습 루프와 같음

100%

마지막 값 60.4% · 합 48.8% · RNN 100%

실행 결과 · 노트북 '학습시켜 보기' 셀



오늘 가져갈 두 번째 문장

순서가 중요하다면 무엇을 넘길 것인가

사람이 정할까 · 학습시킬까

그런데 직접 짜서 돌리면

거의 모두가 여기서 한 번 막힘

코드로 옮기면

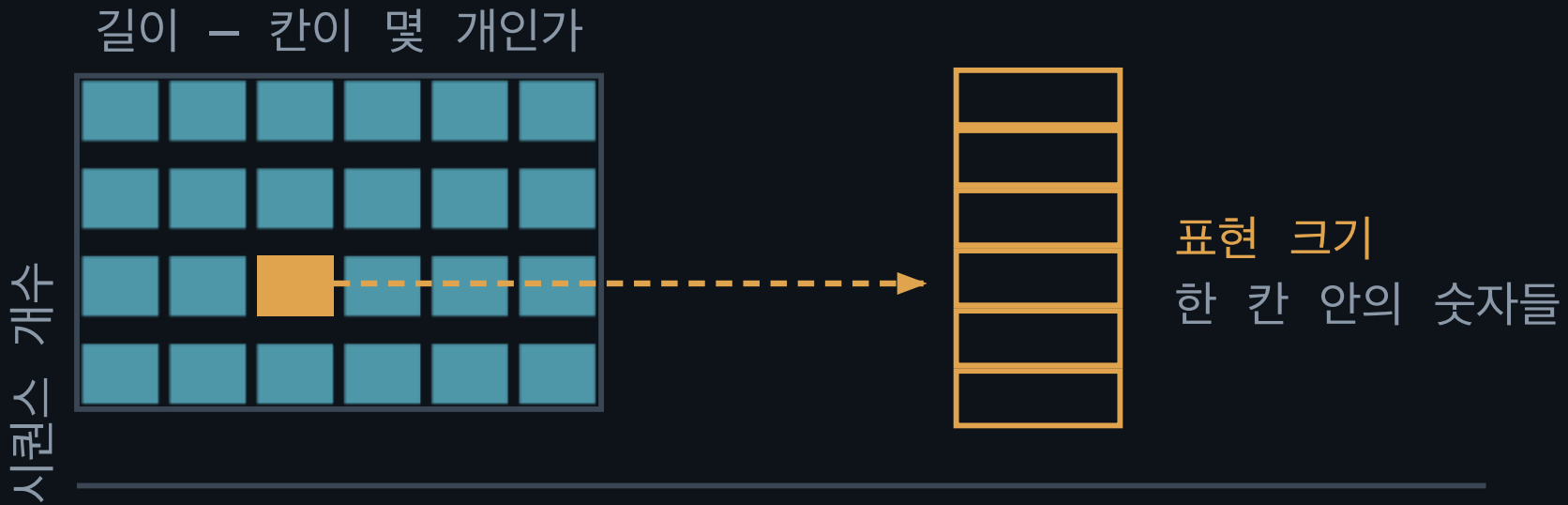
방금 본 에러

여기서 이어감

MLP 에 넣던 모양은 두 겹이었음

- 표 데이터 — 행 개수 · 특징 개수
- 이미지 — 장 수 · 픽셀 (펼치면 두 겹)
- 한 샘플이 숫자 한 줄이었음

시퀀스는 한 샘플이 한 줄이 아님



시퀀스 개수 · 길이 · 표현 크기
batch_first=True 일 때 이 순서임

세 겹이 각각 무엇인가

- 시퀀스 개수 — 한 번에 몇 줄을 처리하는가
- 길이 — 한 줄에 칸이 몇 개인가
- 표현 크기 — 칸 하나를 숫자 몇 개로 적는가

셋째가 제일 헛갈리는 자리임

우리 과제는 어떻게 되는가

9 3 1 0 7 5 8 6 8 4

- 한 칸에 숫자 하나가 들어 있음
- 그런데 0~9 를 열 칸짜리 로 적기로 했음
- 해당 자리만 1, 나머지는 0 임

에러가 난 이유

세 겹은 맞음 · 마지막 겹이 1 임

고치는 법

축을 붙인 게 아니라 칸을 열 칸으로 적은 것임

통과

실행 결과 · 노트북 '모양 맞추기 — one_hot · batch_first' 셀

batch_first 가 정하는 것

- 켜면 — 시퀀스 개수 · 길이 · 표현 크기
- 끄면 — 길이 · 시퀀스 개수 · 표현 크기
- 앞의 두 겹의 순서만 바뀜

PyTorch 의 기본값은 꺼짐임

꼈을 때가 더 무서움

에러가 안 남 · 8 을 길이로, 10 을 줄 수로 바꿔 읽음

실행 결과 · 같은 셀

여기 까지

- 모양이 세 겹임 — 개수 · 길이 · 표현
- 막히면 먼저 모양을 찍어볼 것
- batch_first 는 앞 두 겹의 순서임

돌려주는 것이 두 개였음

- output — 매 칸의 요약을 모은 것
- h_n — 마지막 칸의 요약

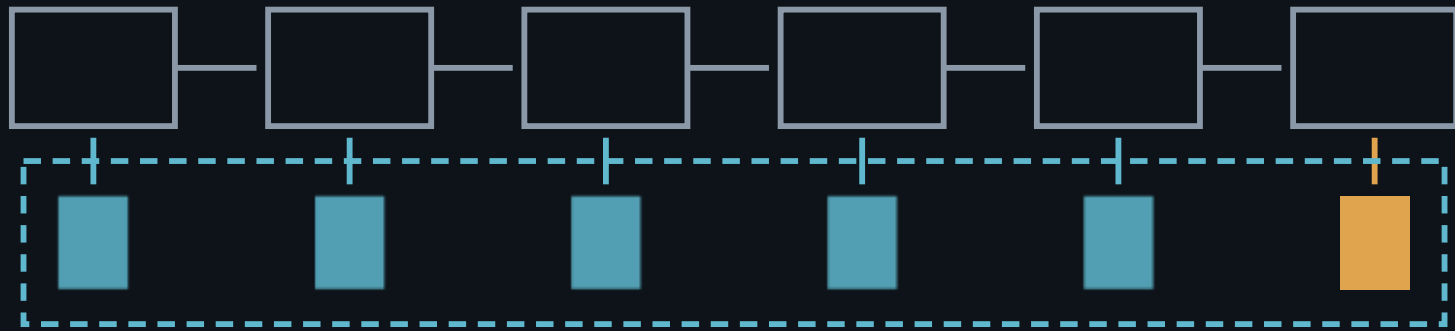
조금 전에 모양은 봤음 · 값이 똑같은지는 아직 안 봤음

모양을 나란히 놓으면

output 은 가운데 10 이 살아 있음 · h_n 은 없음

실행 결과 · 노트북 '모양 맞추기 — one_hot · batch_first' 셀의 batch_first=True 줄

칸마다 계산이 한 번씩 일어남



output - 파란 것 전부

h_n - 주황 하나

마지막 하나는 양쪽에 다 들어 있음

정 말 같은 값 일 까

같은지 다른지 먼저 짐작해 볼 것

비교하는 두 가지 방법

- equal — 비트까지 똑같은가
- allclose — 아주 작은 오차까지 봐주는가

True

`equal` 도 `True` · `allclose` 도 `True`

실행 결과 · 노트북 '정말 같은 값일까' 셀

그대로 쓸 수 있는 조건이 있음

- 층이 하나임 — 여러 층이면 h_n 은 층마다 있음
- 한 방향임 — 양방향이면 뒤로 읽은 것도 있음
- 빈 칸이 없음 — 이때만 -1 이 진짜 마지막 칸임

앞의 둘이 깨지면 $h_n[0]$ 과 값이 달라짐 · 셋째가 깨지면 둘 다 빈 칸을 지난 값임

h_n 앞에 붙은 1 은 무엇인가

- 시퀀스 개수가 아님 · 길이도 아님
- 총 개수 $\times$ 방향 수 임
- 총 하나 · 한 방향 이라 1 임

그럼 왜 돌을 따로 주는가

오후에 하나가 더 늘어남 · 거기서 갈라짐

그래서 무엇을 쓰는가

- 매 칸마다 답이 필요함 → **output**
- 끝에 한 번만 답하면 됨 → **마지막 요약** ($h_n \cdot \text{output}$ 마지막 칸)

한 줄에 라벨이 하나인가 칸마다인가 로 갈림

지금까지는 우리가 만든 데이터였음

- 3 과 7 이 하나씩 · 길이 10 고정
- 정답이 무엇으로 정해지는지 우리가 앎
- 그래서 왜 60% 인가 를 계산할 수 있었음

UCI HAR

- 허리에 찬 폰 센서로 활동 6종을 맞추는 과제
- 걷기 · 계단 오르기 · 계단 내리기 · 앉기 · 서기 · 눕기

UCI HAR Dataset · CC BY 4.0

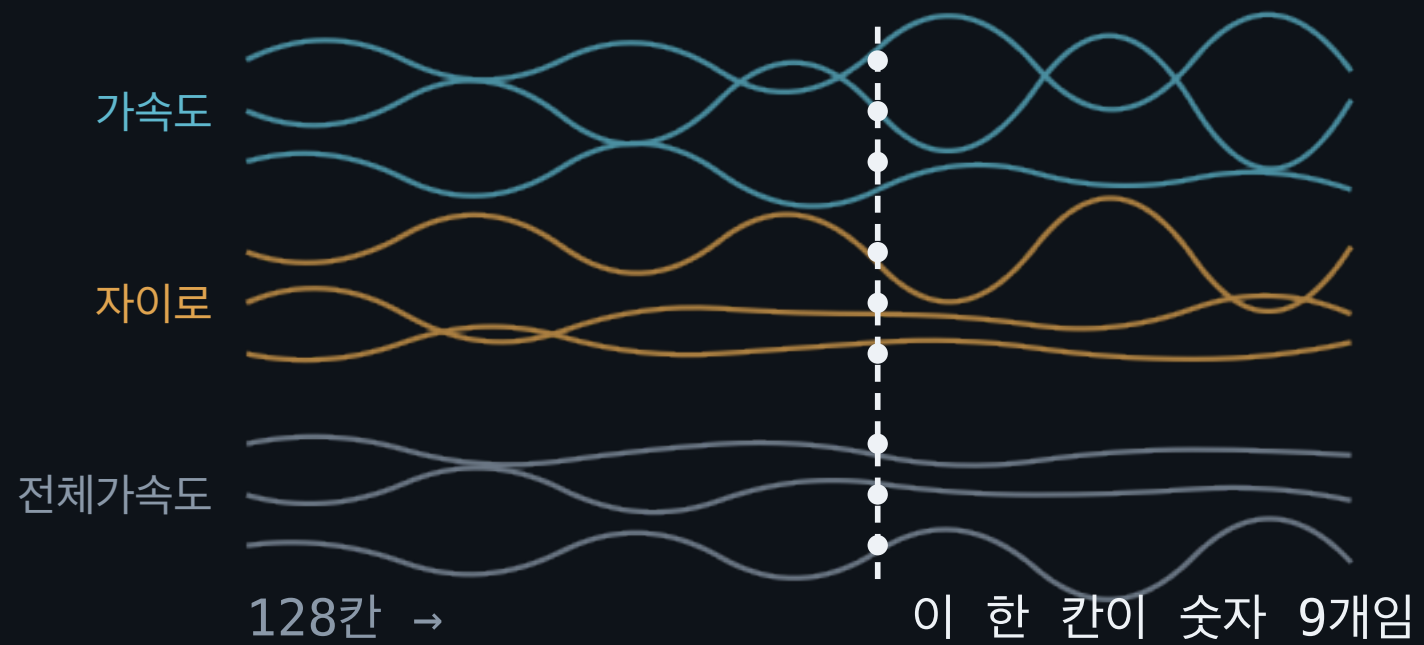
열어보면

실행 결과 · 노트북 'UCI HAR 열어보기' 셀

9 가 어디서 나왔는가

- 가속도 — $x \cdot y \cdot z$
- 자이로 — $x \cdot y \cdot z$
- 중력까지 포함한 가속도 — $x \cdot y \cdot z$

신호 3종 $\times$ 축 3개 = 9 · 한 순간에 동시에 읽음



그런데 128 이 전부 같음

- 7352개 기록이 전부 길이 128 임
- 2.56초씩 잘라서 만든 것이기 때문임
- 그래서 직사각형 배열에 그냥 들어감

진짜 데이터가 늘 이렇지는 않음

그런데 우리 인공 과제는

- 길이를 10 으로 우리가 고정한 것 임
- 길이를 바꾸면 20 도 50 도 200 도 됨
- 한 배치 안에서 제각각으로 만들 수도 있음

HAR 은 이미 잘라놓은 것 · 우리는 자르기 전 을 다룸

HAR 은 오후에 다시 꺼냄

- 지금은 모양만 보고 지나감
- 오후에 RNN · LSTM · GRU 를 여기에 붙임

과제 안내는 오후에 함

순서를 섞어도 답이 같은가
다르다면, 길이도 매번 다른가

두 번째 갈래를 지금 다룸

먼저 그런 데이터를 만들어 봄

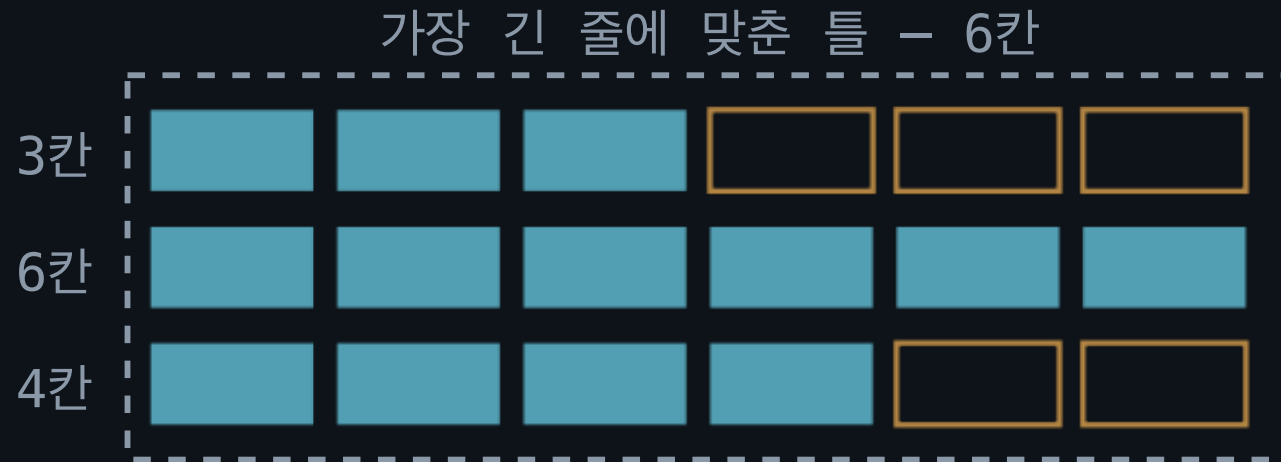
세 줄의 길이가 3 · 6 · 4 로 제각각임

그대로 텐서로 만들면

실행 결과 · 노트북 '그대로 텐서로 만들면' 셀

배열은 직사각형임

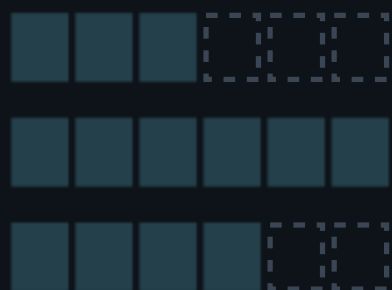
줄마다 칸 수가 다른 배열은 만들 수 없음



주황 칸 다섯 개는 데이터가 아님
그럼 여기를 무엇으로 채울 것인가

틀을 잡는 세 가지 방법

가장 긴 것에 맞춤



빈 칸이 5개 남음

넘치면 자름



뒷부분 2칸을 버림

배치 안에서만



빈 칸이 1개로 줄어듦

억지로 맞추는 이 일을 padding 이라고 함

손으로 채워 보면

모양은 해결됨

실행 결과 · 노트북 '세 줄 채워 보기' 셀

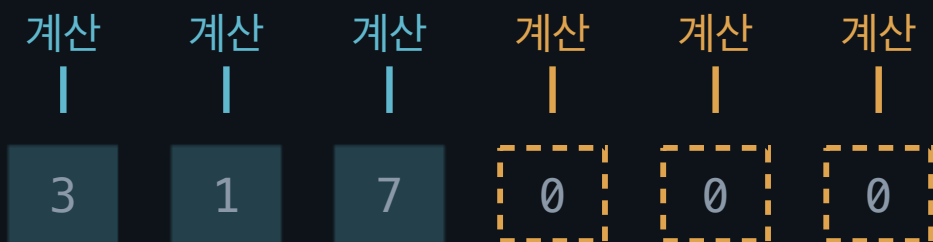
무엇으로 채웠는지가 갈림

- 빈 벡터 — 열 칸이 전부 0 임
- 숫자 0 — 첫 칸만 1 임

실행 결과 · 노트북 '무엇으로 채웠는지' 셀

빈 벡터로 채워도 끝난 게 아님

세 칸짜리 줄을 여섯 칸 틀에 넣은 것임



진짜 값 3칸

0 으로 채운 3칸

모양은 맞았지만 빈 칸을 세 번 더 지남

모양 문제는 풀렸지만 읽는 문제가 남음

그래서 lens 를 같이 들고 다님

- 어디까지가 진짜인지는 X 안에 없음
- 입력에 진짜 0 이 있으면 따로 적어두지 않고는 되찾을 수 없음

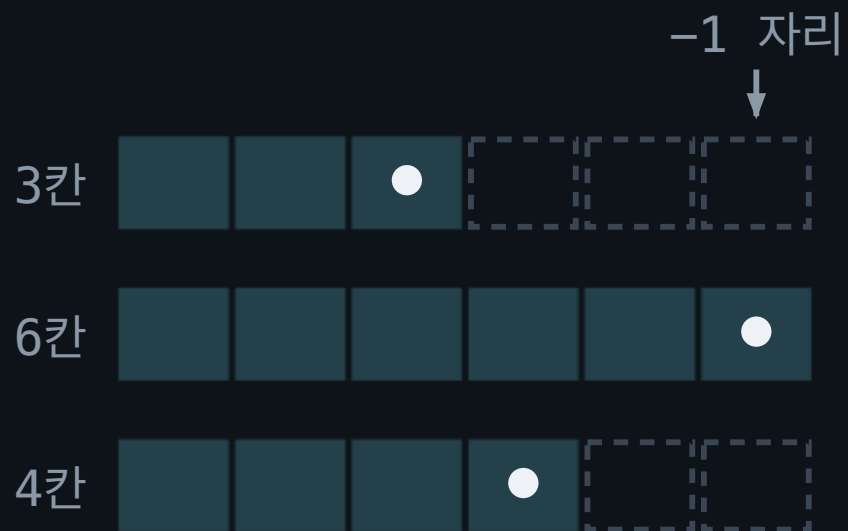
넣는 것은 padding 꺼내는 것은 무엇인가

억지로 맞춘 자리를 계산에서 빼라고 알려주는 것

흔히 쓰는 줄

아까까지는 이게 맞았음

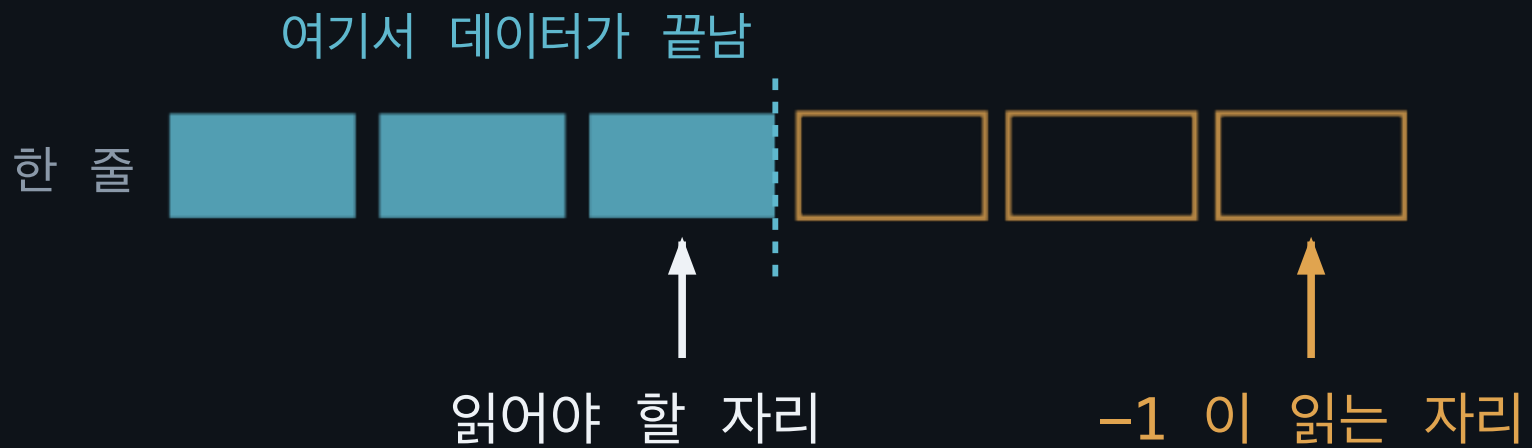
마지막 칸이 무엇인가



이 줄만 -1 이 맞음

● 진짜 마지막 칸 — 줄마다 자리가 다름

-1 은 틀의 끝이지 데이터의 끝이 아님



두 자리가 줄마다 다르게 벌어짐

이대로 돌리면 몇 %일까

길이 5~60 · 길이 10 에서는 100% 였음

예측 ③

▶ 빈 칸을 읽으면 몇 % 인가 · 답 보기
실행 결과 · 길이 5~60 · 노트북 L1b 셀

왜 찍기까지 내려가는가

- 빈 칸 요약에는 답이 남아 있지 않음
- 몇 칸을 더 지났는지가 줄마다 다름
- 모델이 일관되게 틀릴 수도 없음

고치는 줄

- 줄마다 자기 길이의 끝 을 꺼냄
- -1 대신 `len - 1` 임

평균에서 빈 칸을 빼는 법

m 의 첫 줄은 1 1 1 0 0 0 임 . 진짜 칸만 1 임

평균도 똑같이 무너짐

아까 말한 **중간 형태** 가 이것임

실행 결과 · 길이 5~60 · 노트북 L1b 셀

그런데 길이 4~12 로 하면

네 가지가 전부 100% 임 · 차이가 안 남

실행 결과 · 노트북 '길이 4~12 로 좁히면' 셀

편차가 작으면 안 드러남

- 4~12 — 빈 칸이 많아야 여덟 칸임
- 5~60 — 빈 칸이 쉰다섯 칸까지 생김
- 버그는 편차가 커질 때만 보임

이번 시간에 푼 것

길 이 도 매 번 다 른 가

억지로 맞추고 · 맞춘 자리를 빼고 읽음

넘긴 것이 안 닿는다

지금까지 확인한 것

- 순서를 섞으면 답이 달라지는 문제였음
- 넘길 것을 학습으로 정하니 100% 였음
- 모양과 길이를 다루는 법까지 봤음

RNN 이 잘 된 것은 전부 짧은 줄에서 확인한 것임

오후에 물을 것
넘긴 것이
끝까지 닿는가

3 7

9

1

4

0

...

6

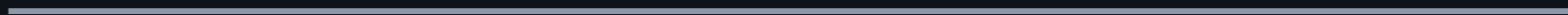
2

8

여기서 답이 정해짐



이 구간을 요약이 건너가야 함



$$\text{거리} = \text{길이} - 2$$

거리 몇에서 무너질까

거리 8 · 48 · 78 · 98 · 198 · 398 을 비교함

예측 ④

먼저 앞쪽 셋

- 거리 8 → 100%
- 거리 48 → 100%
- 거리 78 → 100%

미리 돌린 결과 · 노트북 L2 셀

그리고 뒤쪽 셋

- 거리 98 → 51.7%
- 거리 198 → 50.1%
- 거리 398 → 48.3%

찍기가 50% 임

미리 돌린 결과 · 노트북 L2 셀

78 과 98 사이에서 절벽이 있음

- 78 까지는 하나도 안 틀림
- 98 부터는 아무것도 못 맞힘
- 중간이 거의 없음

학습을 세 배 더 시키면

- 거리 98 → 100%
- 거리 198 → 100%
- 거리 398 → 100%

20 epoch → 60 epoch · 그 외에는 전부 같음

미리 돌린 결과 · 노트북 '학습을 세 배 더 시키면' 셀

방금 실험이 말해준 것

- 거리가 멀어서 못 하는 것이 아니었음 · 덜 배운 것이었음
- 사백 칸도 넘길 수 있었음
- 다만 더 오래 걸렸음

78 까지는 20 epoch 로 됨 · 98 부터는 20 epoch 로 안 됐고 60 epoch 에서 됨

가능한 설명이 둘임

- 요약을 끝까지 **나를 수 없음** — 기억 탓
 - 나를 수는 있는데 그렇게 배우기가 느림 — 학습 탓
- 정확도 하나로는 둘을 못 가름

그래서 실험을 둘로 나눔

- 앞 → 뒤 · 맨 앞을 바꾸면 마지막 요약이 바뀌는가
 - 뒤 → 앞 · 마지막 손실의 신호가 맨 앞까지 오는가
- 두 번 따로 재고 나서 합칠 것

귓속말 전달 게임

- 첫 사람이 한 마디를 옆 사람에게 속삭임
- 옆 사람이 또 옆 사람에게 옮김
- 스무 명을 지나면 말이 달라져 있음

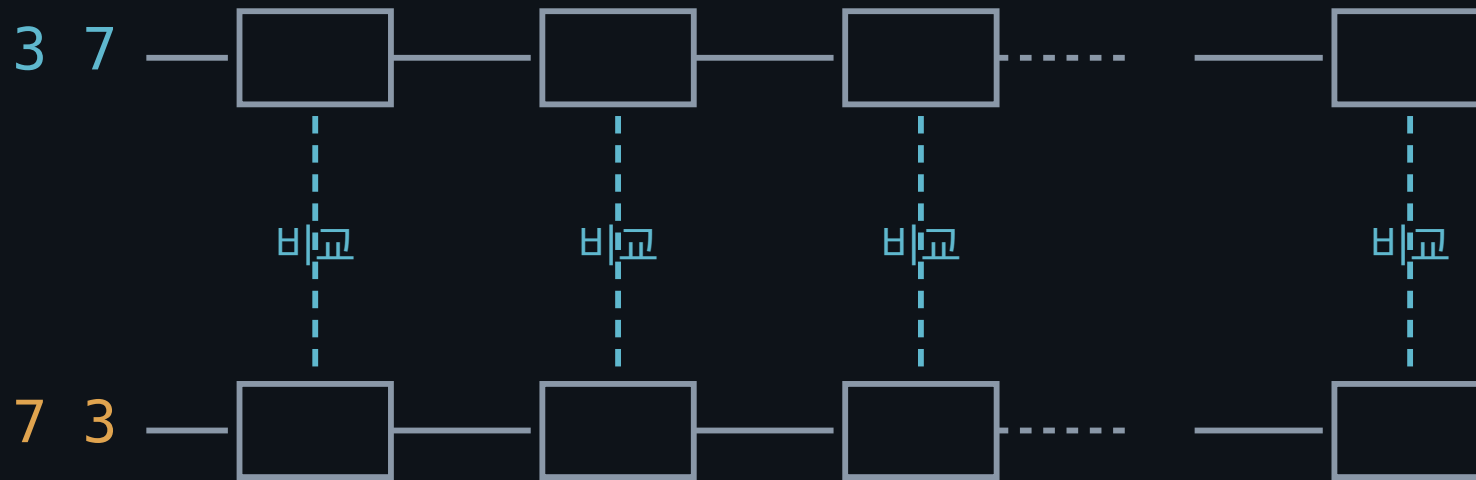
RNN 이 하는 일이 정확히 이 모양임

실험으로 바꾸면

- 줄 두 개를 준비함 · 맨 앞 두 칸만 서로 다름
- 나머지 칸은 글자 하나까지 같음
- 칸마다 두 요약이 얼마나 다른지를 봄

3 7 9 1 4 0 ... 6 2 8

7 3 9 1 4 0 ... 6 2 8



두 요약의 차이가 칸마다 얼마나 남는가

학습 전 모델부터

1칸 0.736

2칸 0.378

5칸 0.061

10칸 0.00255

20칸 0.00000767

40칸 0.000000099

99칸 0.0000000998

실행 결과 · 노트북 '앞 → 뒤 · 학습 전 모델' 셀

스무 칸과 마흔 칸 사이에 바닥에 닿음

- 한 칸 갈 때마다 절반 남짓만 남음
- 열 칸이면 처음의 290분의 1 만 남음
- 마흔 칸이면 두 요약을 구분할 수 없음

0.00000001 이라는 숫자의 정 체

- float32 가 구분할 수 있는 가장 작은 상대 차이가 0.0000000119 임
- 그보다 작으면 '0 에 가깝다' 가 아니라 **젤 수가 없음**

여기는 측정의 바닥임

노트북 '앞 → 뒤 · 학습 전 모델' 셀 마지막 줄

학습시킨 모델 · 51.7% 짜리

1칸 1.73

5칸 0.964

10칸 0.224

20칸 0.0137

40칸 0.000151

99칸 0.000000143

미리 돌린 결과 · 노트북 '앞 → 뒤 · 20 epoch 모델' 셀

학습시킨 모델 · 100% 짜리

1칸 1.73

5칸 1.5

10칸 1.73

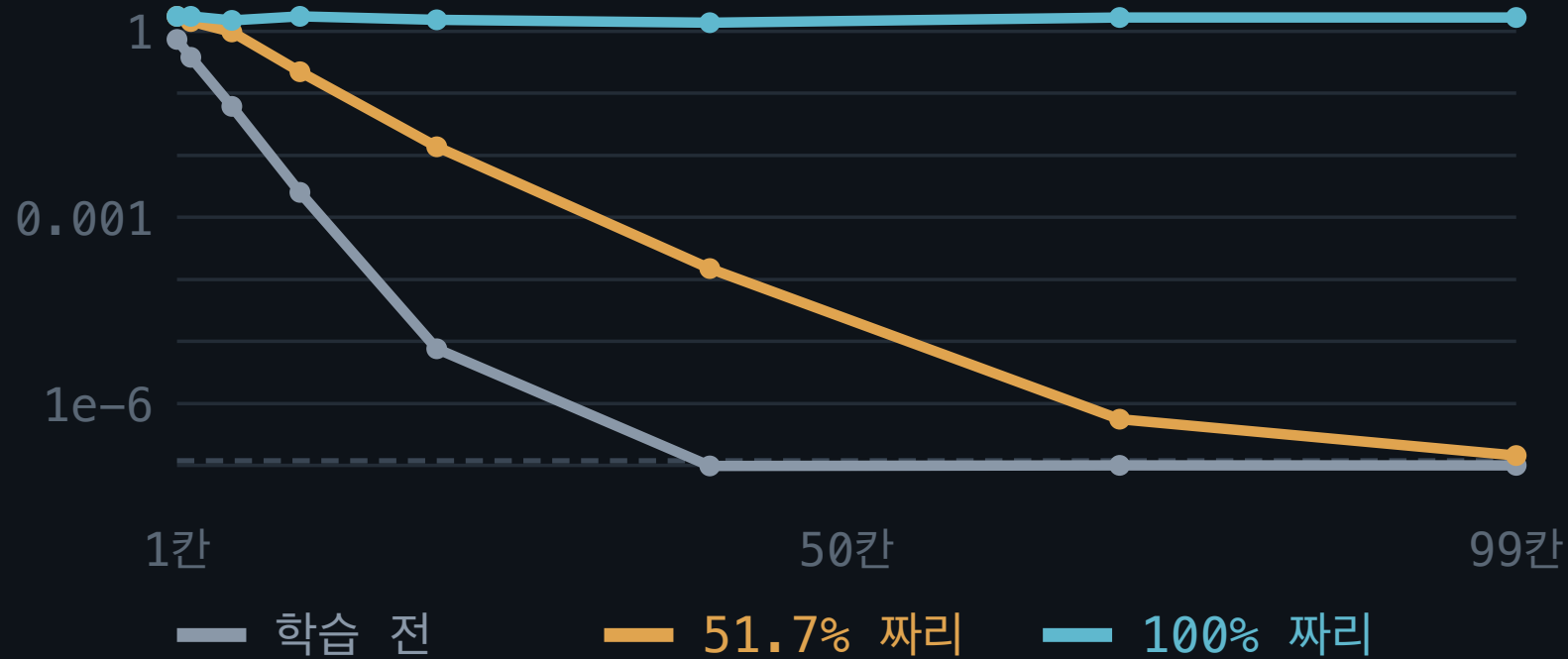
20칸 1.54

40칸 1.37

99칸 1.68

미리 돌린 결과 · 노트북 '앞 → 뒤 · 60 epoch 모델' 셀

셋을 나란히 놓으면



바닥의 점선 아래는 float32 가 구분하지 못하는 자리임

달게 만드는 것이 학습임

학습 없이 손으로 심어봐도

- 이전 요약을 그대로 넘기게 가중치를 박아둠
- 3 이면 $+0.5$ · 7 이면 -0.5 를 한 칸에 적게 함

거리 398 → 100%

끝에 남은 크기 · 거리 8 에서 0.0377 · 거리 398 에서 0.0322

실행 결과 · 노트북 '학습 없이 손으로 심은 가중치' 셀

첫 번째 실험의 답

기억은 달음

단 배우고 나서 달음

그럼 남은 질문

- 거리 78 은 20 epoch 로 배웠음
- 거리 98 은 20 epoch 로 안 됐고 60 epoch 에서 됐음

왜 멀수록 배우기가 느린가

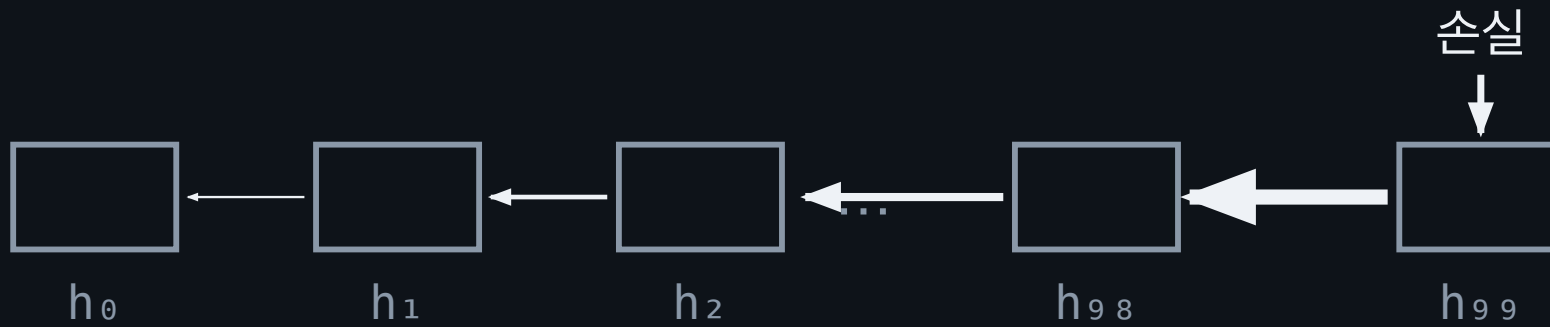
방향을 바꿈

- 답이 틀렸다는 것은 마지막 칸에서 알게 됨
- 그런데 고쳐야 할 곳은 맨 앞 칸임

신호가 백 칸을 거슬러 가야 함

기울기 = 고치라는 신호의 크기

- 이 값을 이만큼 바꾸면 손실이 얼마나 줄어드는가
- 크면 — 여기를 많이 고치라는 뜻임
- 0 에 가까우면 — 고칠 방향을 모른다는 뜻임



거슬러 갈수록 신호가 가늘어짐
칸마다 신호의 크기를 재는 것이 오늘 할 일임

오전에 손으로 짠 **for** 문이 다시 필요함

retain_grad 가 필요한 이유

- 파이토치는 중간 계산의 기울기를 쓰고 바로 버림
- 남겨달라고 미리 말해주는 줄임
- 안 쓰면 `h.grad` 가 `None` 임

L3 셀로 재 봄

- 학습은 안 시킴 · 만들자마자 잤
- 0 · 10 · 20 · 40 · 70 · 99 스텝을 찍음

돌리기 전에 20스텝에서 몇 배 줄어 있을지 짐작해 볼 것

거슬러 간 만큼의 신호 크기

0스텝	$5.16e-02$
10스텝	$1.06e-04$
20스텝	$2.94e-07$
40스텝	$1.33e-12$
70스텝	$1.50e-20$
99스텝	$0.00e+00$

실행 결과 · 노트북 L3 셀 · 길이 100 · 학습 전

배율로 바뀌 읽으면

- 10스텝 거슬러 — 486배 작아짐
- 20스텝 거슬러 — 17만 배
- 40스텝 거슬러 — 390억 배

거리가 두 배면 배율이 제곱으로 커짐

99스텝의 0 은 진짜 0 이 아님

- 77스텝에서 처음 0 으로 찍힘
- 같은 계산을 float64 로 하면 3.38e-28 임
- 0 이 아니라 float32 가 못 적는 것 임

그래도 학습에는 0 과 다를 바 없음

실행 결과 · 노트북 '0 이 진짜 0 일까' 셀

두 실험을 같은 모델끼리 놓으면

- 학습 전 · 앞 → 뒤 99칸 0.00000000998 · 뒤 → 앞 99스텝 3.38e-28(float64) — 둘 다 못 감
- 60 epoch · 앞 → 뒤 99칸 1.68 · 뒤 → 앞 99스텝 1.45e-04 — 둘 다 닿음

학습은 학습 전 모델에서 출발함 · 출발점에서는 맨 앞을 고칠 신호가 거의 안 옴

기울기 소실

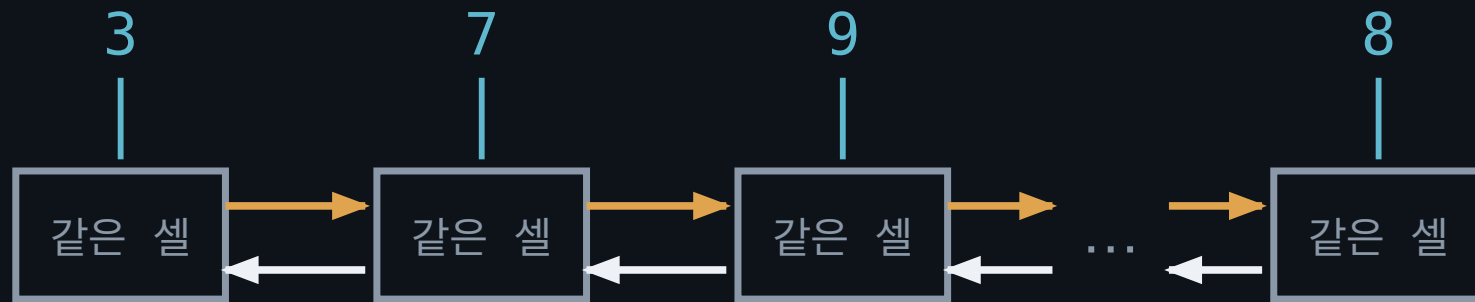
학습 전 · '앞 → 뒤 · 학습 전 모델' 셀과 '0 이 진짜 0 일까' 셀 · 60 epoch · '학습시킨 모델의 기울기' 셀(미리 돌린 결과)

숫자는 봤고 이제 구조를 보

- 신호가 왜 하필 칸을 하나씩 거슬러 가는가
- 이 구조를 알아야 LSTM 이 왜 다른지 보임

역전파를 한 문장으로

- 계산이 지나온 길을 거꾸로 되짚는 것임
- 지나온 길로만 갈 수 있음
- 지름길은 없음



주황 — 앞에서 뒤로 요약이 넘어감

흰색 — 뒤에서 앞으로 신호가 옴

두 화살표가 같은 다리를 씬

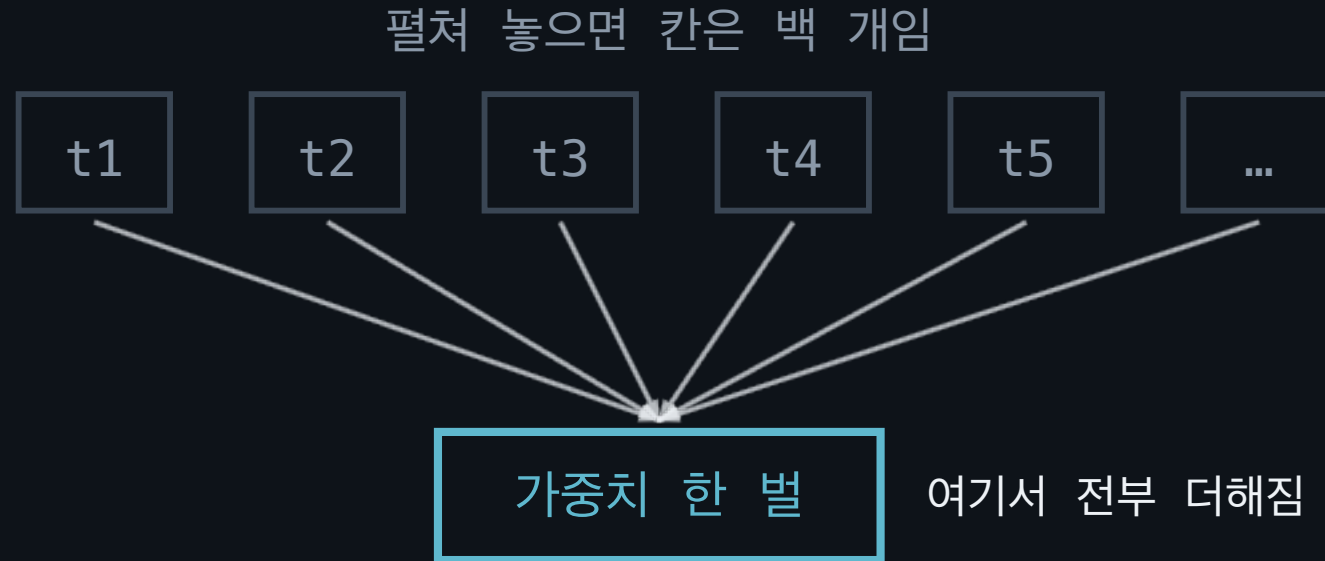
그래서 붙은 이름

BPTT

Backpropagation Through Time

역전파가 시간 축으로도 거슬러 감

파라미터 공유가 여기서 뜻하는 것



- 상자가 하나뿐이므로 고칠 가중치도 하나뿐임
- 백 칸에서 온 신호가 전부 같은 곳으로 더해짐
- 그래서 각 칸의 몫이 작으면 그냥 문힘

그래서 아까 손 코드가 필요했음

- `output` 으로 매 칸의 h 값은 돌려줌
- 다음 칸으로 넘어가는 계산 길 위의 h 는 안에 감춰져 있음

쓰면 안 되는 지름길

이 값은 h 의 기울기가 아님

그래서 아까 표가 믿을 만함

- 손으로 펼친 `nn.RNNCell` 로 잤음
- `h.retain_grad()` 로 중간 요약을 직접 잡았음
- `output.grad` 를 대신 쓰지 않았음

노트북 'output.grad 로 재면' 셀 · 정확히 0 인 칸 99 / 100

여기까지 정리

- 신호는 칸을 하나씩만 거슬러 갈 수 있음
- 백 칸이면 백 번을 거쳐야 함
- 그 백 번 동안 신호가 작아짐

남은 질문 — 왜 하필 작아지는가

한 칸 거슬러 갈 때 생기는 일

- 같은 계산을 거꾸로 한 번 통과함
- 통과할 때마다 크기가 어떤 비율로 바뀜
- 백 칸이면 그 비율이 백 번 곱해짐

다음에 볼 것
같은 수를
계속 곱하면

감을 잡기 위한 계산

- 한 칸 거슬러 갈 때 0.9 배가 된다고 쳐 봄
- 쉰 칸을 거슬러 가면 몇 배가 되는가

암산으로 한 번 해볼 것

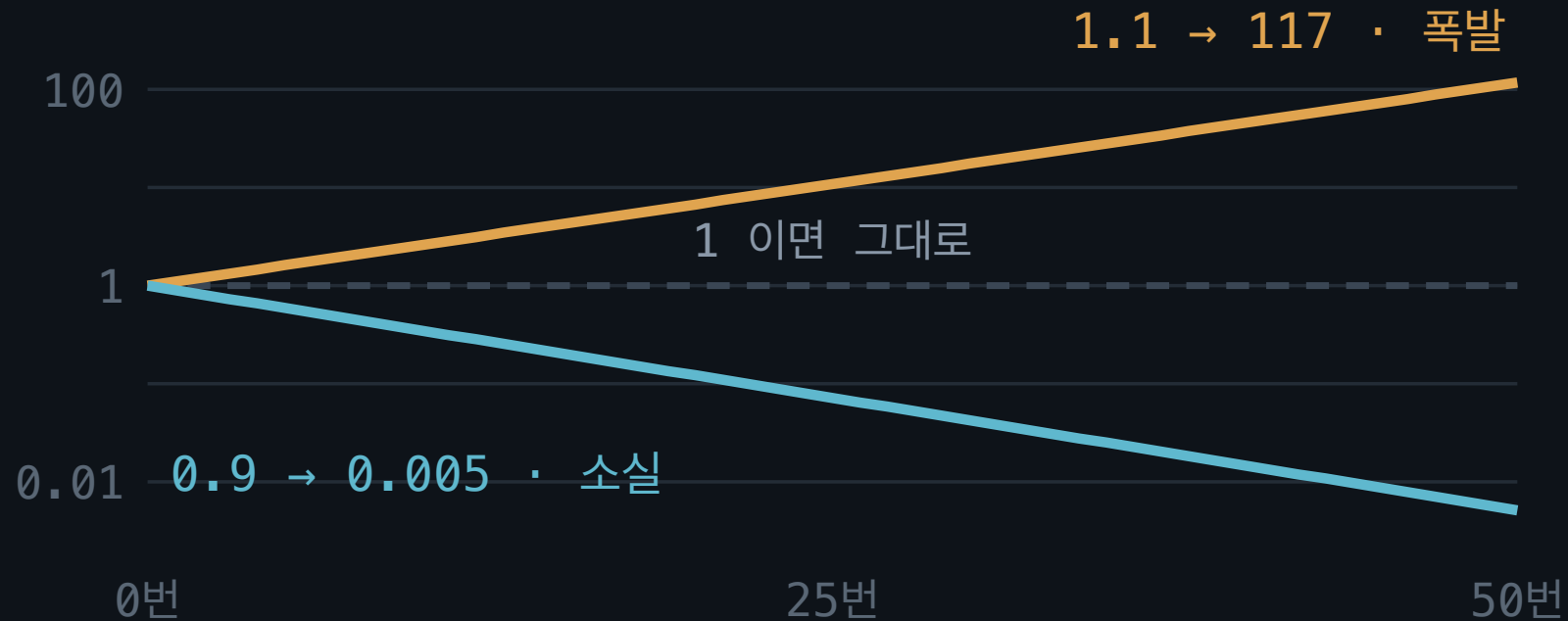
0.9 를 50번 곱하면

0.005

천 분의 5 임

▶ 1.1을 50번 곱하면 얼마인가 · 답 보기

둘은 같은 구조에서 나옴



- 비율이 1 보다 조금 작으면 사라짐 · 조금 크면 터짐
- 딱 1 이어야 유지되는데 그럴 이유가 없음

폭발이 덜 중요한 것이 아님

- 손실이 갑자기 NaN 이 되면 먼저 의심할 것임
- 잘 되던 학습이 한 번에 망가짐
- 다만 눈에 바로 띄

폭발 쪽에는 간단한 대처가 있음

gradient clipping

기울기가 정해둔 크기를 넘으면 잘라내는 것임

▶ 작은 기울기를 통째로 키우면 해결될까 · 답 보기

방금 계산에 단서를 달아야 함

- 실제로 곱해지는 것은 숫자 하나가 아님
- 가중치 행렬이 통째로 반복해서 곱해짐
- 0.9 를 50번 곱하는 그림은 감을 잡는 데까지임

재보면 그림대로 안 감

- 한 번 곱할 때 최대로 늘일 수 있는 배율 1.10
- 같은 행렬을 거듭 곱할 때 남는 배율 0.57
- tanh 미분 평균 0.96 까지 곱하면 0.55 · 실제로 켜 한 칸당 비율 0.54

실행 결과 · 노트북 '가중치 행렬을 재면' 셀

그래도 이 그림을 쓰는 이유

- 왜 거리가 늘면 급격히 나빠지는지가 보임
- 왜 조금만 바뀌어도 크게 달라지는지가 보임

단 답을 낼 때는 재서 낼 것

오후에 한 것

- 거리를 늘려 성능으로 확인했음 — 98 은 20 epoch 로 안 됐고 60 epoch 에서 됐음
 - 학습 전 모델 — 앞 → 뒤도 뒤 → 앞도 99칸을 못 감
 - 60 epoch 모델 — 앞 → 뒤 99칸에서 1.68 · 뒤 → 앞도 달음
- 출발점에서 맨 앞을 고칠 신호가 거의 안 옴 · 느렸던 가장 유력한 이유임

오늘 가져갈 세 번째 문장

넘긴 것이 끝까지 닿는가

못 닿는 게 기억 탓인가 · 학습 탓인가

다음에 할 것

- 신호가 거슬러 올 다른 길을 내주면 어떻게 되는가
- 같은 자로 재서 숫자로 비교할 것

오늘 만든 자를 그대로 씀

남길 것을 학습시키자

조금 전까지 본 것

- 거슬러 가는 기울기를 직접 찍어봄
- 스무 칸만 가도 17만 배 작아졌음
- 배우면 달지만 맨 앞을 고칠 신호가 거의 안 와 느렸음

넘긴 것이 끝까지 닿는가

지금까지 넘기던 방식

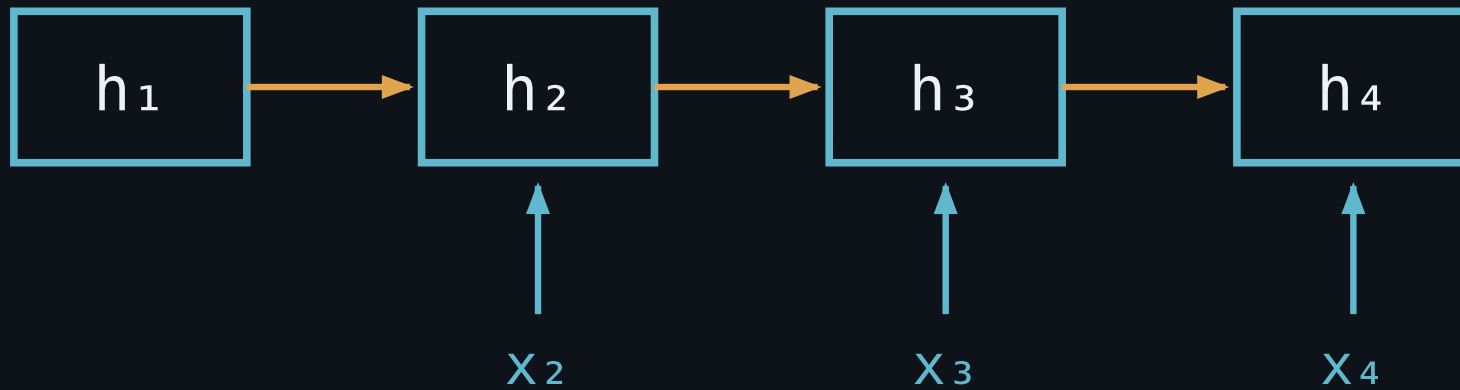
새 요약 = 계산(이전 요약 , 새 입력)

$$h = \tanh(W \cdot x + U \cdot h + b)$$

매 칸에서 h 를 통째로 새로 만듦

직전 h 를 얼마나 남길지 정하는 자리가 없음

화살표 하나가 지날 때마다 내용이 전부 바뀜



얼마나 남길지 고르는 자리가 없음

없는 것

얼마나 통과시킬지

- 매 칸 같은 식으로 섞어 새로 만들 뿐임
- 남길 양을 고르는 중간 손잡이가 없음

필요한 부품 수도꼭지

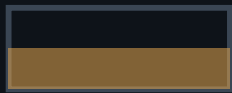
- 잠그면 아무것도 안 지나감
- 반만 열면 절반만 지나감
- 다 열면 전부 지나감
- 여는 정도를 0 과 1 사이 숫자 하나로 적는 것

0.0



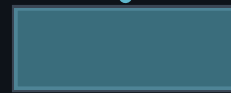
아무것도 안 감

0.5



절반만 감

1.0



전부 감

숫자를 0 과 1 사이로 눌러주는 함수

$f = \text{sigmoid}(W \cdot x + U \cdot h + b)$

- 안쪽 계산은 어떤 값이든 나올 수 있음
- sigmoid 를 씌우면 반드시 0 과 1 사이가 됨
- 그래서 여는 정도로 바로 쓸 수 있음



쓰는 법은 곱하기 하나임

통과한 양 = $f \times$ 원래 값

f 가 0 이면 아무것도 안 지나감

f 가 1 이면 그대로 지나감

f 를 사람이 정하지 않고 학습으로 정하는 것이 핵심임

부품은 생겼음
어디에 달 것인가

h 하나에 달면 되는지부터 따져볼 것

h 하나가 맡고 있는 일

- 지금 답을 내는 데 쓰임
- 다음 칸으로 넘어감
- 백 칸 뒤까지 살아남아야 함

한 줄이 세 가지를 동시에 맡고 있음

회의를 한 시간 듣는다고 해볼 것

- 노트에 적는 것 — 나중까지 필요한 것만 적음
- 지금 입으로 대답하는 것 — 방금 들은 것에 대한 반응임
- 들은 양도 다르고 바뀌는 속도도 다름

노트를 매 문장마다 찢지는 않음

같은 회의를 듣지만 하는 일이 다름

길게 남길 메모

세 번만 적힘



거의 그대로 흘러감



지금 꺼내 쓸 말

매 순간 바뀜

한 줄이 버거운 이유

- 답을 잘 내려면 방금 것에 민감해야 함
- 멀리 보내려면 방금 것에 둔감해야 함
- 한 줄에 두 요구가 같이 걸려 있음

한 줄로도 닿기는 했음 · 두 요구가 한 자리에서 부딪히는 것임

그래서 줄을 하나 더 뒀

- cell state — 길게 남길 메모
- hidden state — 지금 꺼내 쓸 말

이름이 낫선 것뿐임 · 하는 일은 방금 본 그대로임

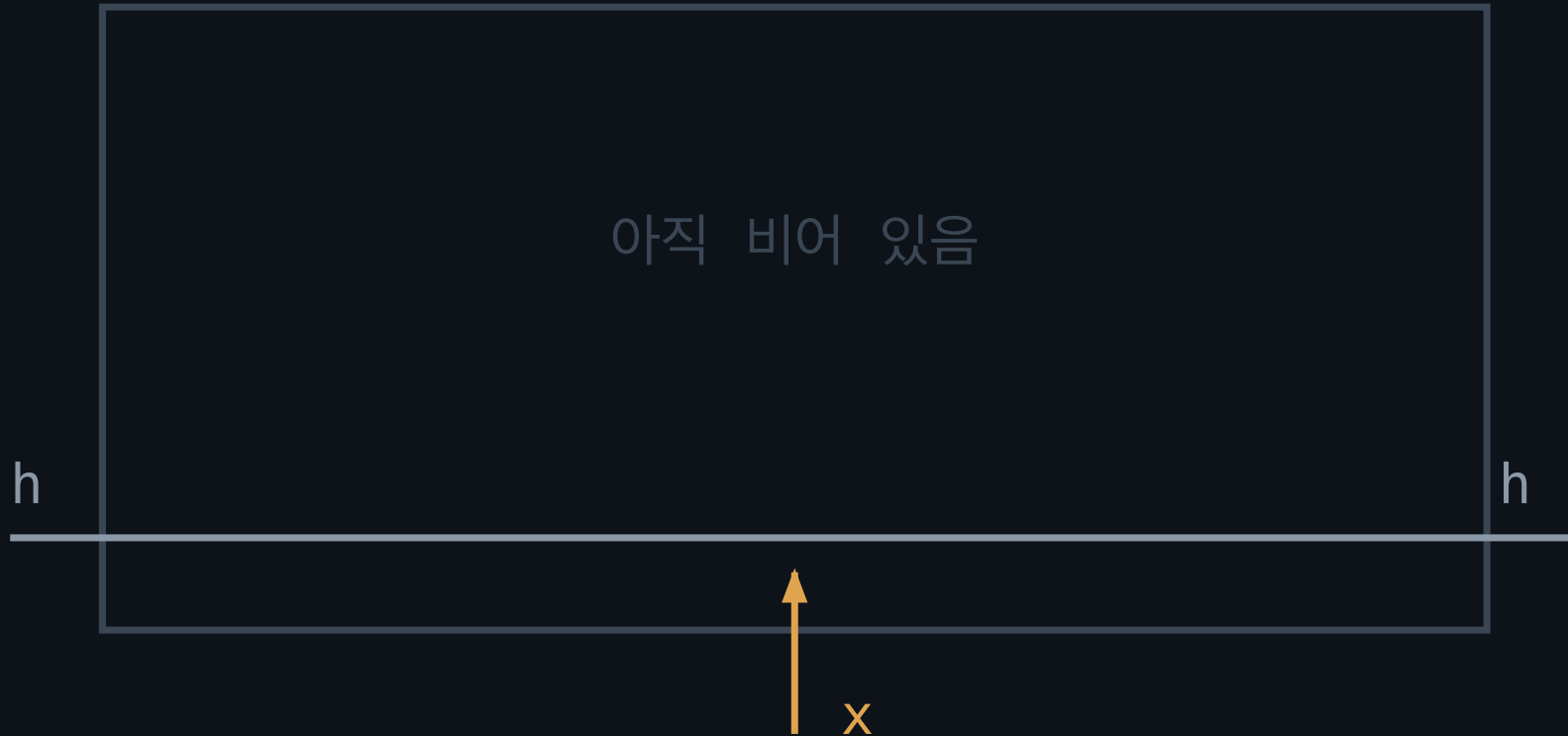
메모에 허용된 일은 두 가지 뿐임

- 일부를 지우는 것 — 곱하기
- 새로 덧붙이는 것 — 더하기

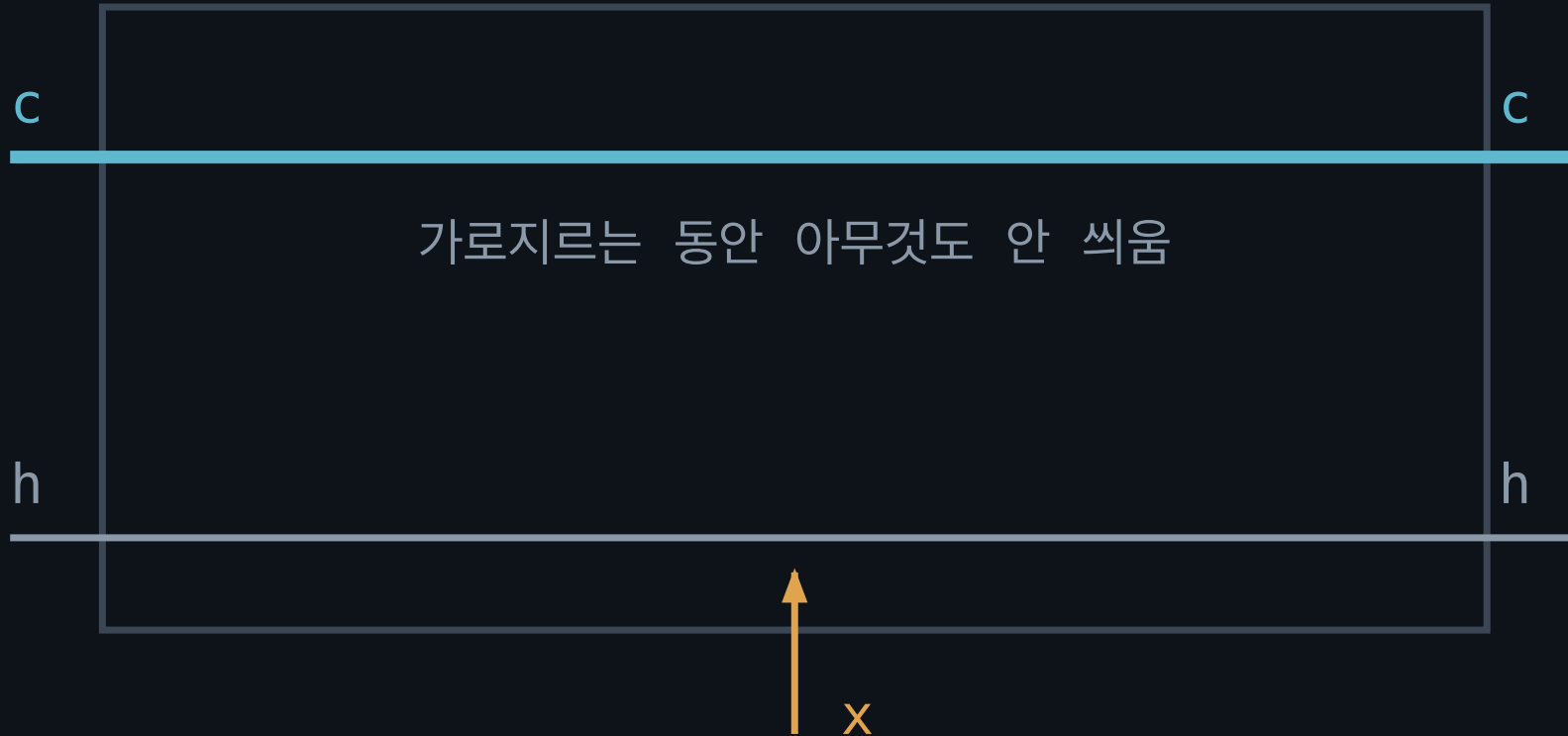
통째로 다시 쓰는 일은 없음

$$c = (\text{지울 만큼}) \times c + (\text{덧붙일 것})$$

한 칸 안에서 벌어지는 일임



새로 생긴 것은 위쪽 줄 하나뿐임



가로지르는 동안 아무것도 안 씹음

직선 하나가 하는 일

- 거슬러 가는 길이 곧게 뚫려 있음
- 매 칸에서 무언가를 곱하지 않음

점심 뒤에 본 '매 칸 곱해지며 줄어듦' 이 없는 구조임

줄은 놓았음
언제 지우고
언제 덧붙일 것인가

사람이 정할지 학습시킬지 · 오전과 같은 질문임

메모에 붙일 손잡이

- 메모에서 지울 만큼을 정하는 문
- 메모에 새로 적을 만큼을 정하는 문
- 메모에서 지금 꺼낼 만큼을 정하는 문

첫 번째 · forget gate

메모를 얼마나 남길 것인가

$$C \leftarrow f \times C$$

f 가 1 이면 그대로 남김 · 0 이면 지움

이름은 잊는 문이지만 실제로 정하는 건 남기는 양 임

얼마나 남길지 정하는 문이 붙음



두 번째 · **input gate**

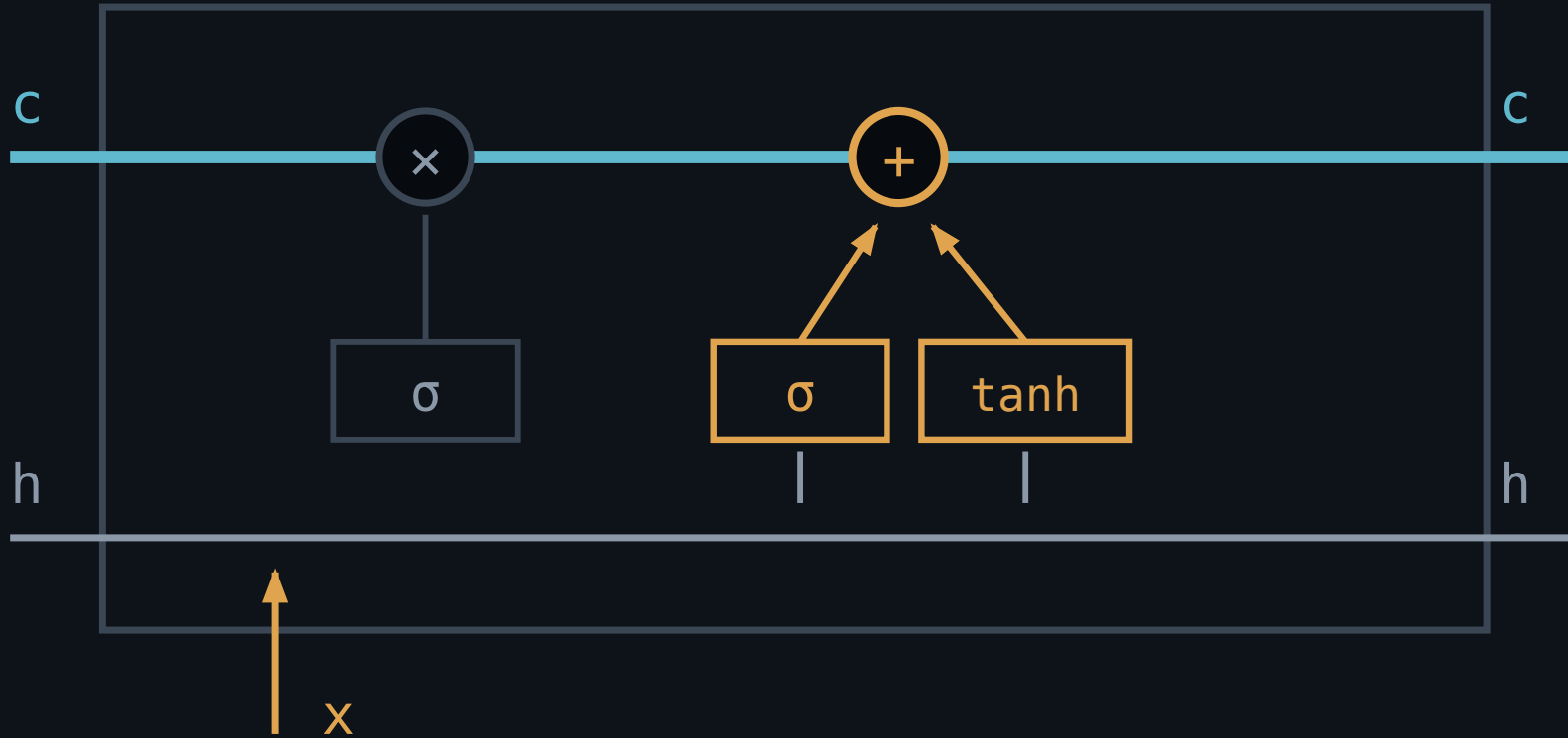
무엇을 얼마나 적을 것인가

$$c \leftarrow c + i \times g$$

g 는 적을 내용임 · $\tanh$ 로 만듦

i 는 그중 얼마나 적을지임 · sigmoid 로 만듦

적을 내용과 적을 양이 따로 만들어짐



세 번째 · output gate

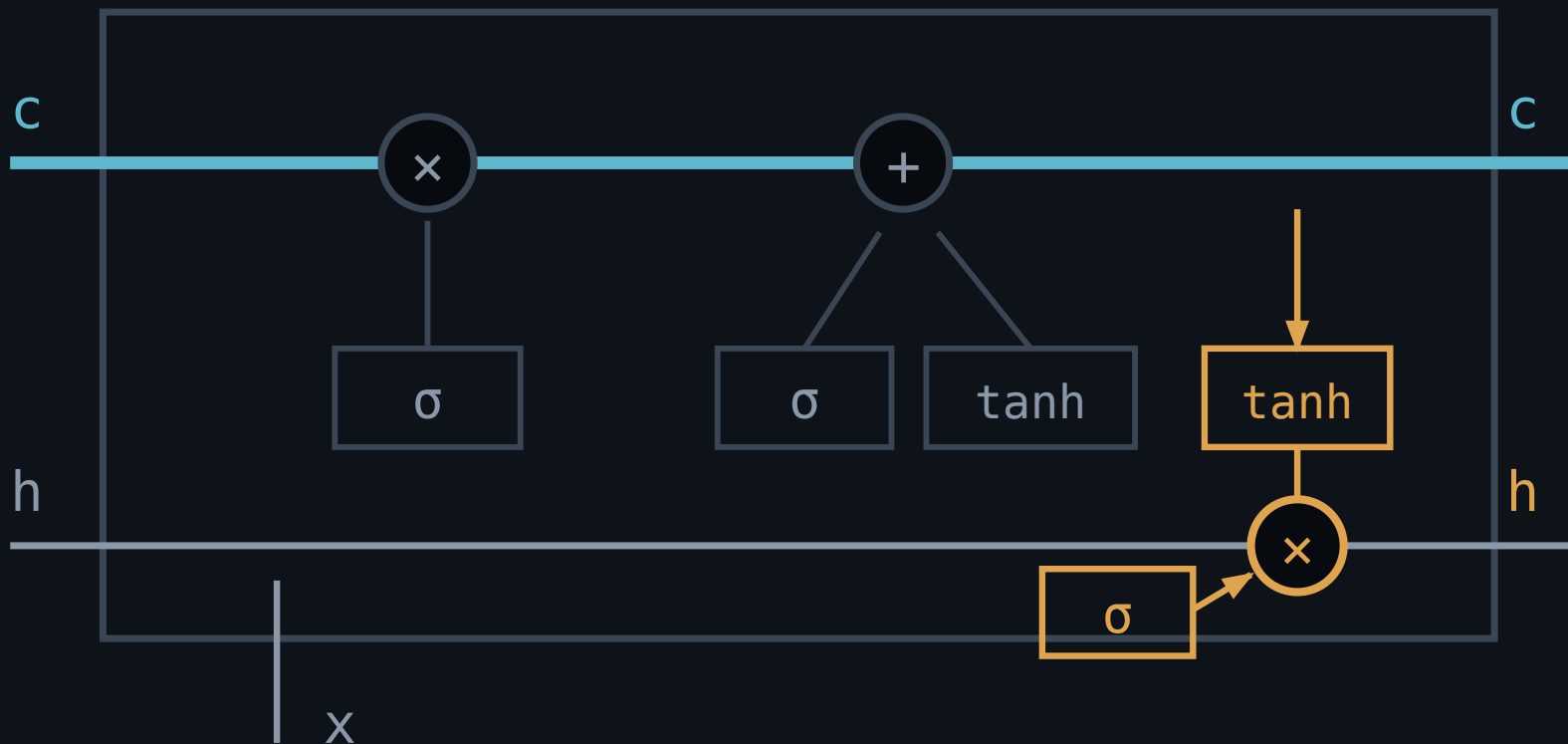
메모에서 지금 얼마를 꺼낼 것인가

$h \leftarrow o \times \tanh(c)$

메모 전체를 매번 꺼내지는 않음

여기서 h 와 c 가 서로 다른 값이 됨

메모에서 꺼낸 만큼이 h 가 됨



세 문이 정하는 것

- forget — 메모에서 남길 양
- input — 메모에 적을 양
- output — 메모에서 꺼낼 양

셋 다 0 과 1 사이 값임 · 셋 다 학습으로 정해짐

코드에서는 한 줄임

▶ LSTM은 몇 개의 텐서를 돌려줄까 · 답 보기
실행 결과 · 노트북 'LSTM 이 돌려주는 것' 셀

값은 치르고 있음

- 문 세 개와 적을 내용까지 네 벌을 계산함
 - 같은 hidden 크기에서 파라미터가 크게 늘어남
- 늘어난 값이 성능으로 돌아오는지는 나중에 숫자로 볼 것

그런데
정말 살아날까

아까 죽은 기울기를 다시 찍어볼 것

아까 찍은 그 표

스무 칸에서 이미 천만분의 셋임

소수 한 자리로 반올림해 다시 적은 것임

조건을 똑같이 맞춤

- 같은 데이터 · 같은 길이 100 · 같은 hidden 32
- 같은 손실 · 같은 방식으로 기울기를 찍음
- 바꾼 것은 셀 종류 하나뿐임

문을 여는 정도는 0 주변 에서 출발함

따로 열어주지 않은 보통의 LSTM 임

아흔아홉 칸 뒤에서 기울기가 살아 있을까

RNN 은 0이었음

살아 있음 / 죽어 있음 중 하나를 고를 것

예측 ⑤

▶ LSTM 은 99칸 뒤에 기울기가 남는가 · 답 보기
실행 결과 · 노트북 L3b 셀

나란히 놓으면

10칸 자리에서는 LSTM 이 오히려 더 작음

곧게 뚫려 있다고 했었음

- 메모 줄에는 곱해지는 게 없다고 했음
- 그런데 방금 문 하나를 달았음

$$c \leftarrow f \times c$$

f 가 1 일 때만 곱이 없는 것임

문에 더하는 값이 0 이면 반쯤 닫혀 있음



아까 똑같은 걸 봤음

0.9 를 50번 곱하면 0.005

0.5 를 100번 곱하면 10^{-30} 쪽임

실제 문 값 평균 0.4997 · 백 번을 곱해도 10^{-30} 이었음 · 노트북 '문이 얼마나 열려 있었나' 셀

이름만 다르지 같은 일임

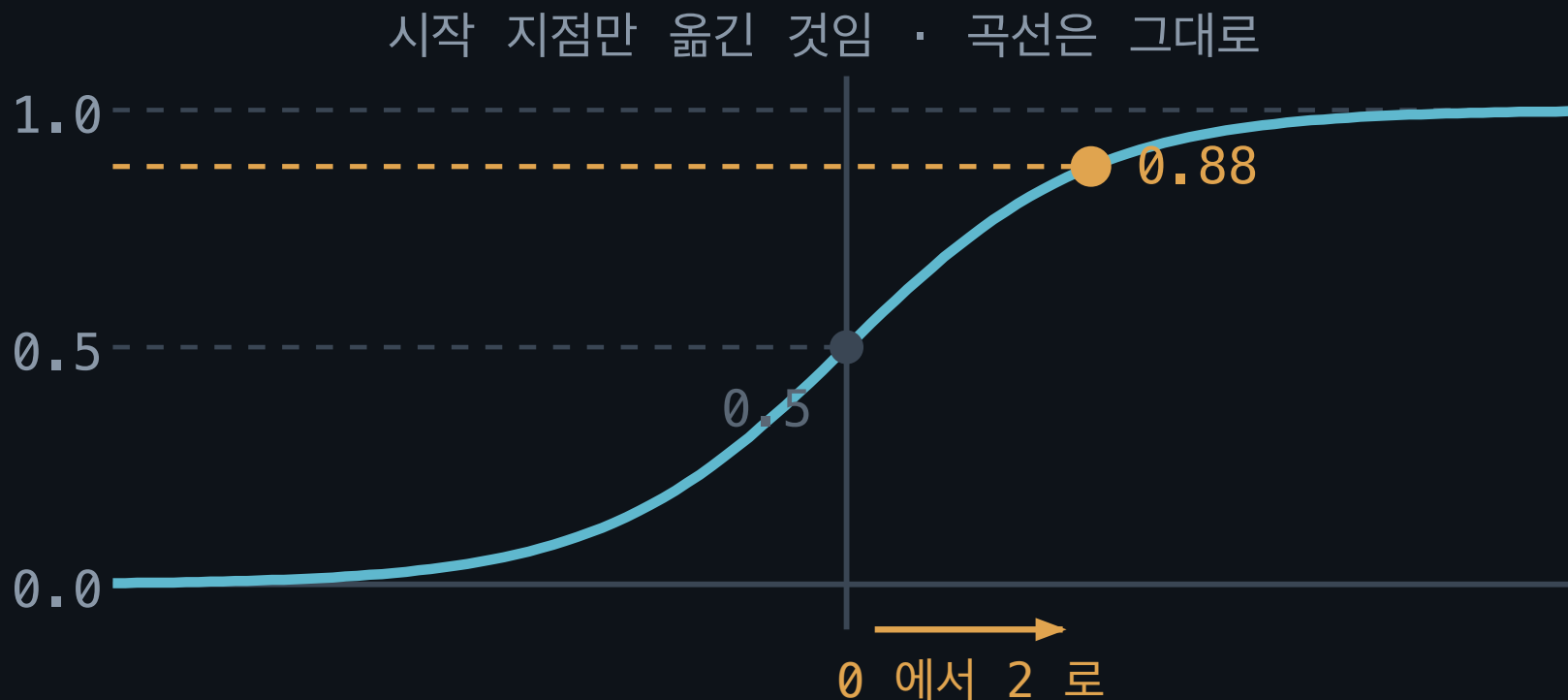
같은 구조가 두 번 나옴

아까 · RNN
0.9 를 **50번**
0.005

방금 · LSTM 기본
0.5 를 **100번**
 10^{-30} 쪽

1 보다 작은 수를 반복해 곱한 것임

그럼 문을 열어두고 시작해 볼 것



실제 백 칸 평균은 0.8779 였음 · 고친 것은 시작값 하나뿐임

문 이름으로는 못 찾음

두 번째 구간이 forget 임

문을 열어둔 경우

99칸 뒤에도 $1.6e-05$ 로 남아 있음

넷째 줄 · 마지막 줄은 곧 봄

실행 결과 · 노트북 '문을 열어두면' 셀

더 열면 더 남음

멀어질수록 오히려 커지기도 함

실행 결과 · 노트북 '문을 열어두면' 셀 넷째 줄

그래서 무엇 때문에 살아난 것인가

- 문이 있어서 살아난 게 아님 — 문에 더하는 값 0 에서도 죽었음
- 문이 열려 있을 때 살아남
- 그리고 언제 열고 닫을지를 학습으로 정할 수 있음

RNN 에는 그 손잡이 자체가 없었음

오늘 가져갈 세 번째 문장

넘긴 것이 끝까지 닿는가

못 닿는 게 기억 탓인지 학습 탓인지 나눠 볼 것

여기까지는 만든 문제였음
실제 데이터에서도
이득이 있을까

폰을 차고 걸어다닌 기록



이 방향으로 7,352 장

개수 · 길이 · 칸마다 값
[7352, 128, 9]

오전에 본 세 겹 그대로임

오전 실습은 길이 10이었음

활동 6가지를 맞히는 문제임 · 그림은 학습용 · 평가용은 [2947, 128, 9]

UCI HAR Dataset · CC BY 4.0

둘 중 어느 쪽이 이길까

RNN · LSTM · 둘 다 hidden 64 로 맞춤

같은 hidden 64

차이가 8%p 넘게 남

미리 돌린 결과 · 노트북 L4 셀

그런데 공정한 비교인가

- LSTM 은 네 벌을 계산하므로 파라미터가 더 많음
- 같은 hidden 64 는 같은 크기가 아님

RNN 5,190개 · LSTM 19,590개

8%p 를 설명하는 두 후보

- 문을 달아서 — 구조 덕
- 파라미터가 네 배 가까이 많아서 — 크기 덕

지금 숫자로는 둘을 가릴 수 없음

지금 말할 수 있는 것과 없는 것

- 말할 수 있음 — 같은 hidden 64 에서 LSTM 이 8%p 넘게 앞섬
- 아직 모름 — 크기 덕인지 구조 덕인지
- 아직 모름 — RNN 이 왜 졌는지
- 아직 모름 — 다시 돌리면 얼마나 흔들리는지

둘 다 한 번씩만 돌린 값임

자주 듣는 말 하나

LSTM 이 기울기 소실을
해결했음

오늘 세 번 어긋난 자리가 있었음

어긋난 세 자리

- 문에 더하는 값을 0 으로 두면 RNN 과 똑같이 0 으로 찍혔음 — 학습에는 0 과 같음
- 10칸 자리에서는 오히려 더 작았음
- 열어둔 경우도 99칸에서 3,303배 줄었음

사라지진 않았지만 크게 줄기는 함

그래서 이렇게 말하는 것이 정확함
해결한 것이 아니라
크게 늦춘 것임

그리고 늦추는 정도를 학습으로 정하게 한 것임

바로 이어서 볼 것

- 8%p 넘는 차이는 크기 덕인가 구조 덕인가
- 손잡이는 꼭 셋이어야 하는가

문을 단 쪽이 크게 앞섰음 · 파라미터는 네 배 가까이 많았음

더 적은 손잡이

방금까지 본 것

- 손잡이 세 개로 나눠서 정했음
- 길게 남길 메모를 따로 한 줄 뒀음

같은 문제를 다르게 푼 답이 하나 더 있음

LSTM 이 나눠 놓은 것

- 무엇을 버릴지 정하는 손잡이
- 무엇을 새로 넣을지 정하는 손잡이
- 무엇을 꺼내 보여줄지 정하는 손잡이
- 길게 남길 메모는 위로 따로 흐름

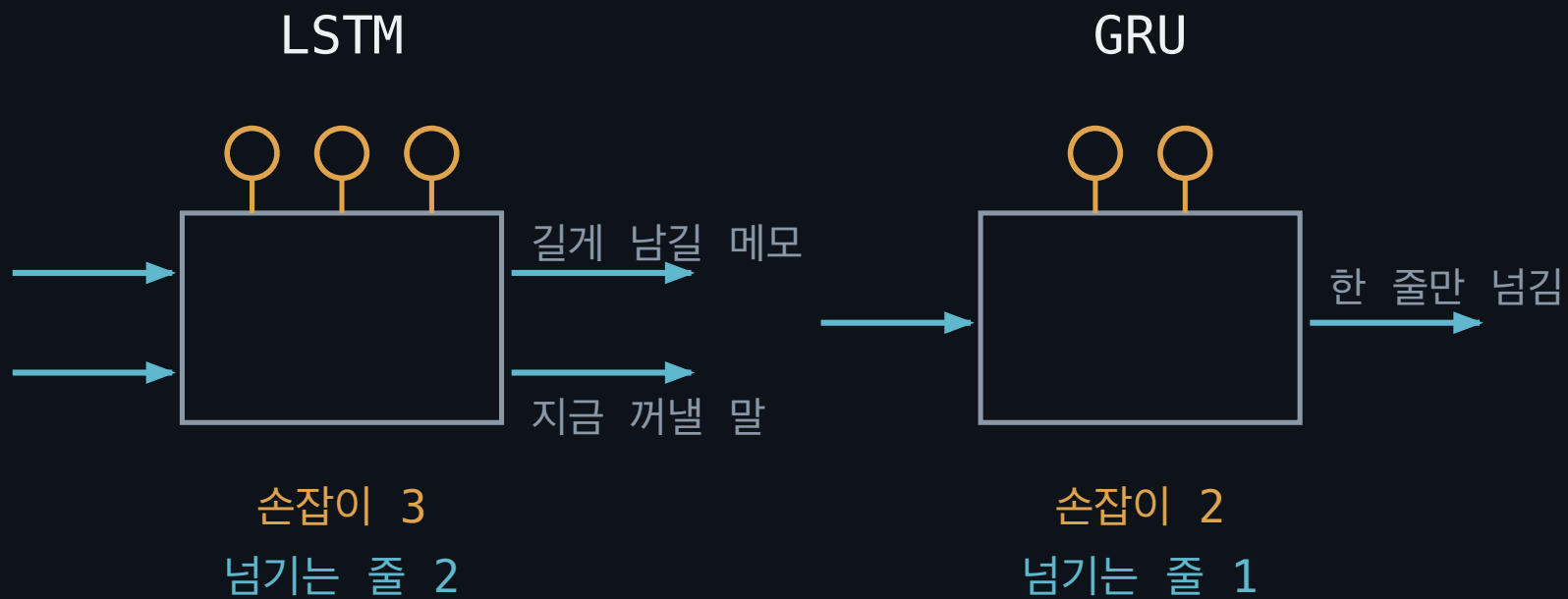
손잡이 셋 · 넘기는 줄 둘

물어볼 것

- 손잡이 세 개가 다 필요한가
 - 버릴 양과 넣을 양이 따로여야 하는가
- 자유로운 만큼 값을 치르는 것임

손잡이를 둘로 줄인 답 같은 문제를 손잡이 두 개로

- 버릴 양과 넣을 양을 한 손잡이가 같이 정함
- 남기는 양과 채우는 양을 합하면 1 임
- 따로 둔 메모가 없음



메모를 따로 두지 않는다는 것

- LSTM — 길게 남길 메모와 지금 꺼낼 말이 두 줄임
- GRU — 둘이 한 줄에 같이 담김

구조만 놓고 보면 할 수 있는 일이 줄어든 것임

이 선택이 맞바꾸는 것

- 버린 것 — 버릴 양과 넣을 양을 따로 정할 자유
- 버린 것 — 길게 남길 메모를 따로 지킬 자리
- 얻은 것 — 곱할 것이 줄어들음

자유를 주고 가벼움을 산 것임

셋에서 둘로 · 둘에서 하나로

LSTM 손잡이 3 · 넘기는 줄 2

GRU 손잡이 2 · 넘기는 줄 1

외울 것은 이 두 줄뿐임

줄여도 된다고 본 이유

- 비우는 양과 채우는 양은 대체로 같이 움직인다고 봄
- 많이 잊는 순간이 새 정보가 많이 드는 순간임
- 그렇다면 한 손잡이로 묶어도 잃을 것이 적음

여기까지는 짐작임 · 숫자가 판정할 것

어느 쪽이 더 잘 맞힐까

손잡이 3 · 줄 2 손잡이 2 · 줄 1

재는 방법을 두 번에 나눌 것

시험대

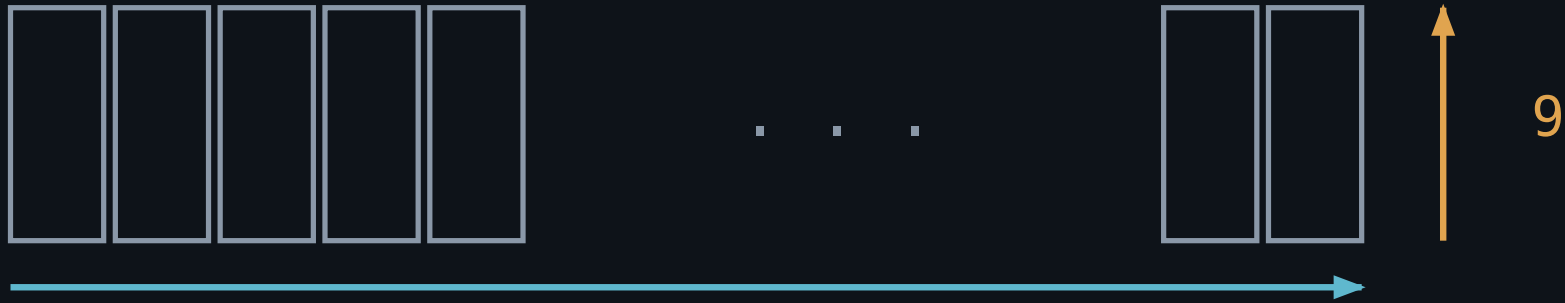
- 폰 센서 값만 보고 활동 6종을 맞추는 문제임
- 진짜 사람이 걷고 앉고 계단 오르는 기록임

UCI HAR Dataset · CC BY 4.0

왜 데이터를 바꾸는가

- 만든 문제는 잘하면 100% 가 나옴
- 다 100 이면 모델 사이가 안 벌어짐
- 아무도 100 을 못 받는 문제가 필요함

학습 7,352 줄 · 평가 2,947 줄



한 줄이 128 칸임

각 칸에 센서 값이 9 개씩 들어 있음

첫 번째 비교 — 구조만 바꾼 비교

- 요약본 크기를 64 로 못 박음
- 가운데 계산만 RNN · LSTM · GRU 로 갈아 끼움
- 데이터 · 학습 설정 · 뒤 분류기는 전부 같음

한 번에 하나만 바꾸는 것임

GRU 는 LSTM 에 비해 어떨까

LSTM 은 90.6% 였음 · 차이는 몇 퍼센트포인트일까
예측

82.5%

RNN · 요약본 64 · 찍기는 17% 근처임

같은 조건의 LSTM 은 90.6% 였음

LSTM 90.6%

GRU 90.1%

RNN 은 82.5% 였음

미리 돌린 결과 · 노트북 'GRU 도 같은 조건으로' 셀

첫 번째 간격

RNN → LSTM 82.5% → 90.6% 8.1%p

RNN → GRU 82.5% → 90.1% 7.6%p

- 100 개 중 17~18 개 틀리던 것이 9~10 개로 줄어듦

- 이 데이터에서 손잡이를 단 두 모델이 같이 움직인 폭임

왜 그만큼인지는 이 데이터에서 재지 않았음

두 번째 간격

$$90.6\% - 90.1\% = 0.5\%p$$

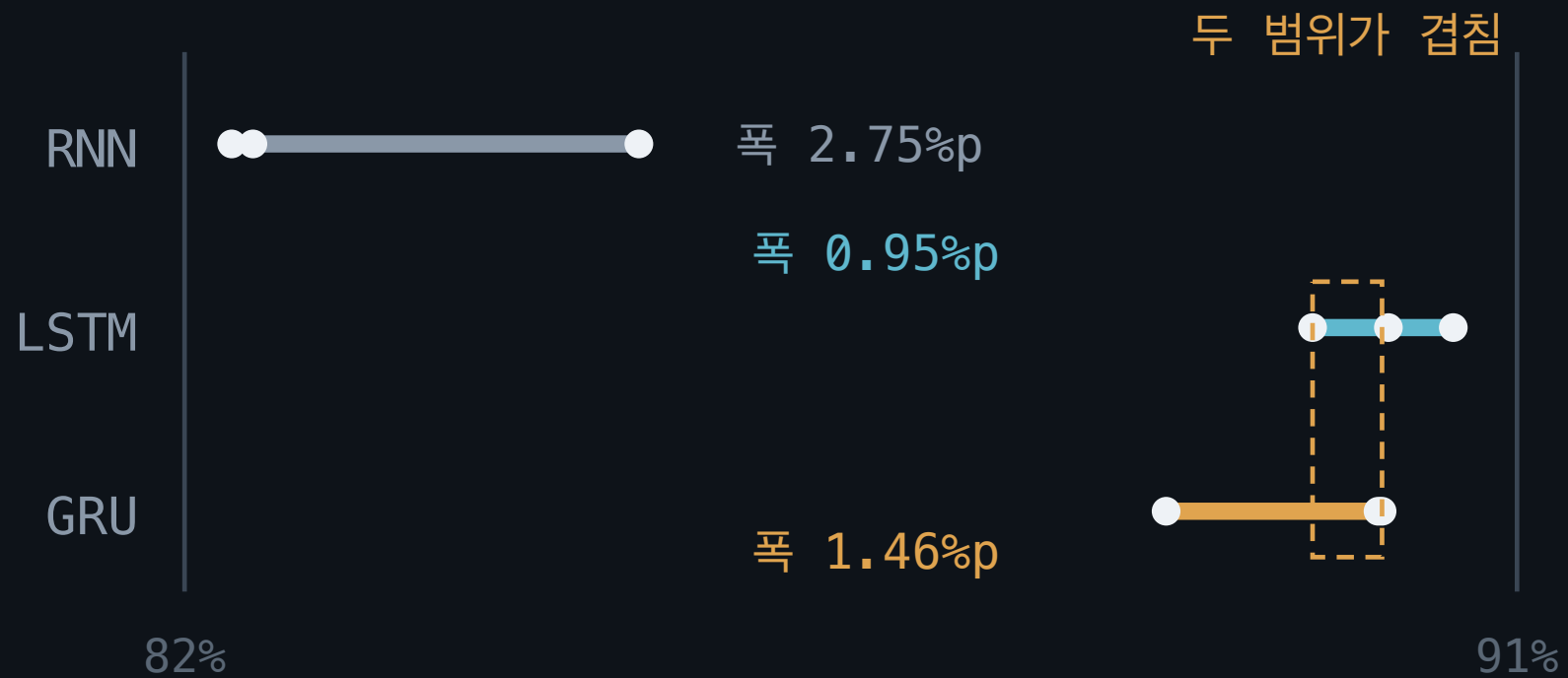
- 평가 2,947 개 중 15 개쯤의 차이임
- 실패가 아니라 알고 싶었던 답임

차이가 거의 없다는 것이 결과임

이 결과가 막아주는 문장

- “LSTM 이 GRU 보다 낫다” — 이 데이터로는 쓸 수 없음
- “GRU 가 LSTM 보다 낫다” — 이것도 쓸 수 없음

구조가 다르다고
성능이 다른 건 아님



시드만 바꿔 세 번씩 돌린 것임 · 미리 돌린 결과 · 노트북 '시드를 세 번 바꾸면' 셀

구조만 바꾼 비교 — 정확도

RNN 82.5%

LSTM 90.6%

GRU 90.1%

위와 아래는 멀고 · 아래 둘은 붙어 있음

같은 화면에 파라미터를 더 함

RNN	82.5%	5,190
-----	-------	-------

LSTM	90.6%	19,590
------	-------	--------

GRU	90.1%	14,790
-----	-------	--------

요약본은 셋 다 64 였음

이 비교가 공정한가

- 같은 64 라도 안에서 계산하는 벌 수가 다름
- RNN 한 벌 · GRU 세 벌 · LSTM 네 벌

hidden 크기가 같다고 공정한 비교가 아님

두 번째 비교 — 조건을 맞춘 비교

- 요약본 크기가 아니라 파라미터 수를 맞춤
- 적게 쓰는 쪽의 요약본을 키움
- RNN 은 128 로 · GRU 는 74 로

맞춘 뒤의 예산

RNN	h128	18,566
-----	------	--------

GRU	h74	19,320
-----	-----	--------

LSTM	h64	19,590
------	-----	--------

RNN 은 5,190 에서 3.5 배 넘게 늘린 것임

실행 결과 · 노트북 '파라미터 수를 비슷하게 맞추면' 셀

3.5 배를 준 RNN 은 얼마까지 갈까

앞에서는 82.5% 였음

예측

83.2%

5,190 → 18,566 개 · 오른 폭은 0.7%p

RNN 이 시드만 바뀌 흔들린 폭 2.75%p 안임

미리 돌린 결과 · 노트북 '맞춘 크기로 다시 돌리면' 셀 첫 줄

크기를 늘려서 되는 문제는 아니었음

- 파라미터를 3.5 배 넘게 줘도 오른 폭은 0.7%p 임
- LSTM 과의 간격은 8.1%p 에서 7.4%p 로 거의 그대로임

RNN 이 왜 졌는지는 이 데이터에서 재지 않았음

GRU 90.8%

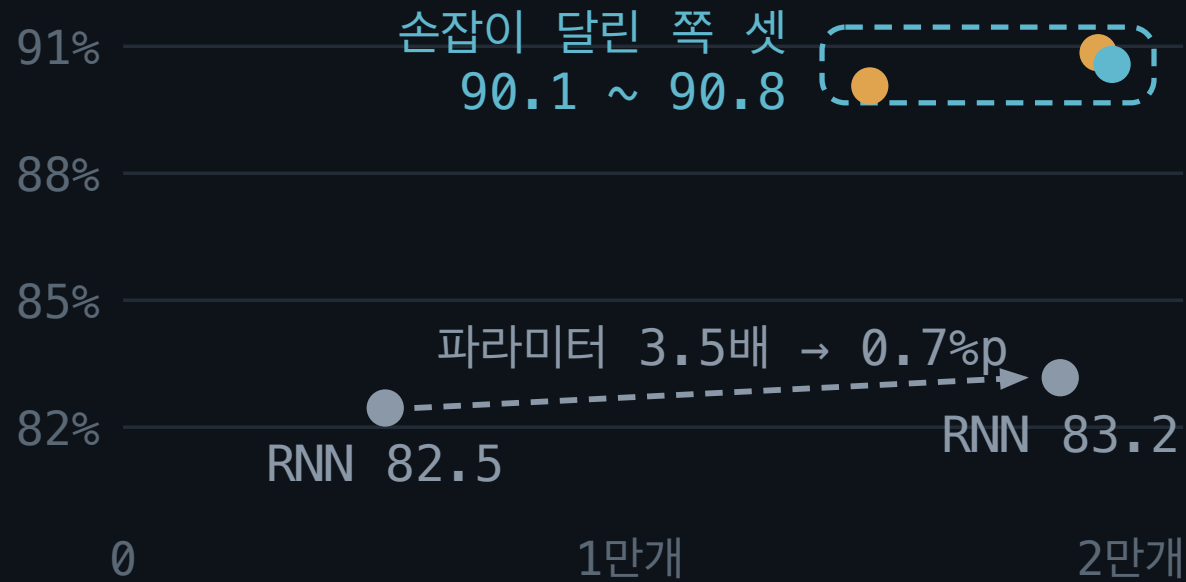
LSTM 90.6%

앞에서는 LSTM 이 0.5%p 위였음

이번엔 GRU 가 0.3%p 위임

미리 돌린 결과 · 노트북 '맞춘 크기로 다시 돌리면' 셀 둘째 줄

두 비교를 같이 놓음



두 번 다르게 잰는데 같은 모양임

미리 돌린 결과 · 노트북 L4 셀 · 'GRU 도 같은 조건으로' 셀 · '맞춘 크기로 다시 돌리면' 셀

파라미터를 맞추면 공정한가

RNN h128 18,566 · 20.1초

GRU h74 19,320 · 36.8초

LSTM h64 19,590 · 52.4초

부품 수가 같아도 걸리는 시간은 같지 않음

노트북 L4 셀 · '맞춘 크기로 다시 돌리면' 셀 · 시간은 제작 맥 기준 · 환경마다 다름

그럼 무엇으로 고르는가

- 정확도로는 못 고름 — 두 번 다 1%p 아래였음
- 남는 기준은 파라미터 수와 시간임

성능이 아니라 비용임

같은 64 에서의 계산서

LSTM 19,590 · 52.4초 · 90.6%

GRU 14,790 · 40.3초 · 90.1%

파라미터 약 25% 적음 · 12.1초 짧음

정확도 차이는 0.5%p 임

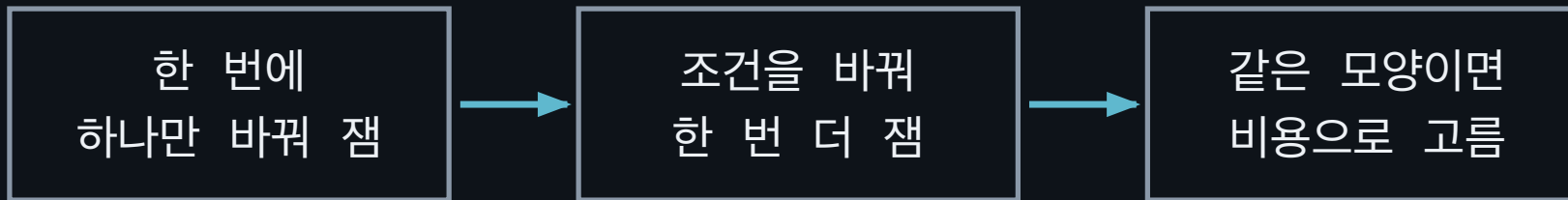
노트북 L4 셀 · 'GRU 도 같은 조건으로' 셀

시간은 제작 맥 기준 · 환경마다 다름

다음에 쓰실 순서

- RNN 으로 먼저 돌려볼 것 — 가장 싸고 빠름
- 안 되면 손잡이 달린 쪽으로 갈 것
- 여유가 없으면 GRU 부터 할 것

차이가 작게 나와도 실험이 잘못된 것은 아님



구조만 바꾼 비교

조건을 맞춘 비교

파라미터 · 시간

모델 둘을 놓고 고민할 때마다 쓰는 순서임

오늘 가져갈 마지막 문장

구조가 다르다고 성능이 다른 건 아님

8%p 와 0.5%p · 갈림길은 앞쪽이었음

오늘 배울 구조는 여기까지임

- RNN · LSTM · GRU 셋을 봤음
- LSTM 과 GRU 사이에서는 성능이 아니라 비용으로 고름

마지막 시간에는 새로 배우는 것이 없음

남는 것

마지막 30분

새로 배우는 것은 거의 없음

새 데이터를 만나면 이 순서로 물음

- 1 한 샘플이 무엇인지 정함 · 길이는 고정인가 가변인가
- 2 순서를 섞어봄 · 답이 달라지는가
- 3 달라진다면 무엇을 기억해야 하는지 적음
- 4 넘길 것을 사람이 정해봄 · 어디서 깨지는가
- 5 깨지면 넘길 것을 학습으로 정하게 바꿈
- 6 입력과 출력의 모양을 정함
- 7 거리를 늘려가며 정말 닿는지 성능으로 확인함
- 8 안 닿으면 기억 문제인지 학습 문제인지 나눠 봄
- 9 문(gate)이 달린 것으로 바꾸고 통제된 비교로 확인함

1단계 · 2단계

- 한 샘플이 무엇인지 정함
- 순서를 섞어봄

오늘 어디였을까

9 3 1 0 7 5 8 6 8 4

9 7 1 0 3 5 8 6 8 4

- 합도 평균도 같음 · 답만 뒤집힘
- 합으로 풀면 48.8% · 찍기보다도 낮음

2단계 — 실제 데이터에서도 해봄

- 폰 센서 기록의 시간 축을 섞음
- LSTM · 시드 3번 평균
- 그대로 → 90.1%
- 섞음 → 86.5%

떨어진 폭 3.6%p · 무너지지는 않음

미리 돌린 결과 · 노트북 '한 줄 안의 시간 순서를 섞으면' 셀

3단계

무엇을 기억해야 하는지 적어보는 것

오늘 어디였을까

3단계 — 오전

- 3 을 봤는가 · 7 을 봤는가 → 51.2%
- 3 도 7 도 늘 둘 다 들어 있었음
- 어느 쪽을 먼저 봤는가 · 상태 셋 → 100%

4단계 · 5단계 — 오전

- 마지막 값 하나만 → 60.4%
- 전부 넘기기 → 길어지면 모델이 커져 무너짐
- 크기만 정하고 안은 학습에 맡김 → 100%

파라미터 1,474개

6단계 — 점심 전

- 텐서가 두 겹에서 세 겹이 됨
- 몇 개 · 몇 칸 · 칸마다 몇 개
- `output` 과 `h_n` 이 뭐가 다른지 돌려서 봄

같은 자리에서 본 것

- 길이가 매번 다르면 억지로 맞춰야 함
- 채운 자리를 그냥 두고 마지막 칸을 읽음

51%

7단계 · 8단계 — 오후

- 학습 전에는 앞 → 뒤도 뒤 → 앞도 99칸을 못 감
 - 학습하면 둘 다 달음 · 20 epoch 로는 안 되고 60 epoch 에서 됨 (lr 3e-3)
 - 맨 앞을 고치라는 신호가 거의 안 와서 느렸을 가능성이 큼
- 0.9 를 50번 곱하면 0.005 · 실측은 한 칸 0.54

9단계 — 오후 끝

- 문에 더하는 값을 0 으로 둔 LSTM 도 죽었음
- 문을 열어야 살았음
- HAR · hidden 64 · RNN 82.5% → LSTM 90.6%
- 파라미터를 맞춰도 간격 7.4%p
- LSTM 과 GRU 는 0.5%p 차이

아홉 단계 · 오늘의 장면

- 1 한 샘플 · 길이는 고정인가
- 2 순서를 섞어봄
- 3 무엇을 기억할지 적음
- 4 넘길 것을 사람이 정해봄
- 5 학습으로 정하게 바꿈
- 6 입력과 출력의 모양
- 7 거리를 늘려 확인함
- 8 기억 타인가 학습 타인가
- 9 문 달린 것 · 통제된 비교

숫자 열 개 한 줄

합으로 50%

51.2% → 상태 셋

60.4%

100% · 1,474개

세 겹 · 51%

60 epoch 에 닿음

0.9 를 50번

82.5 → 90.6

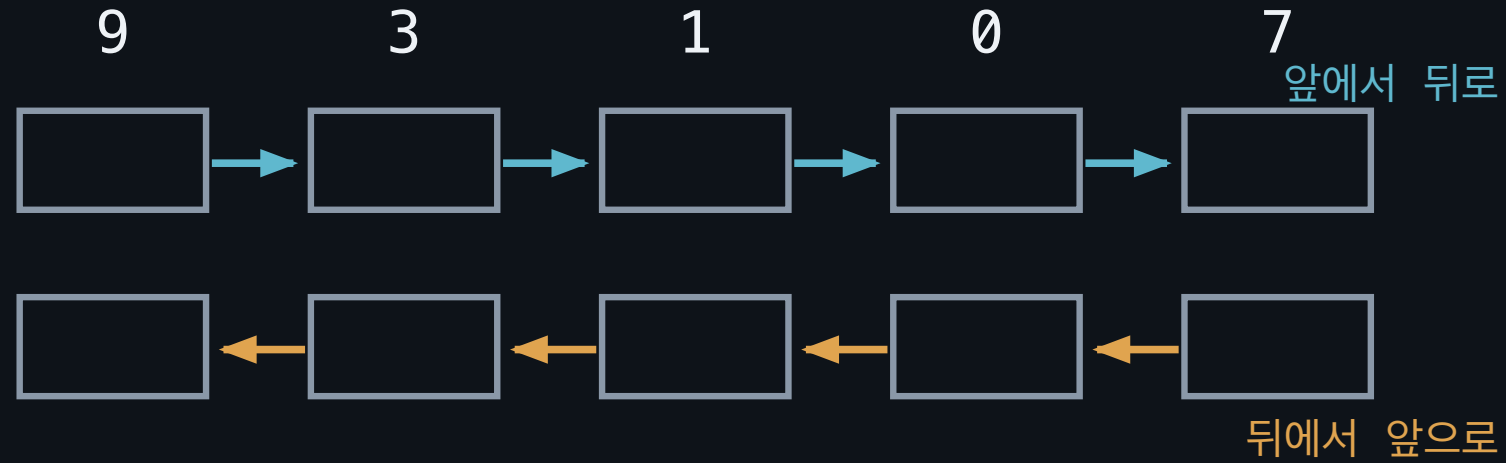
쓰는 법
막히면
그 번호가 지금 자리임

우리는 늘 한 방향으로만 읽었음

왜 그래야 하는지 물어본 적 없음

9 3 1 0 7 5 8 6 8 4

오른쪽부터 읽으면 답이 달라지는가



같은 줄을 두 번 읽음 · 방향만 다름
칸마다 앞에서 온 요약과 뒤에서 온 요약을 가짐

이렇게 읽는 것에 붙은 이름

Bidirectional

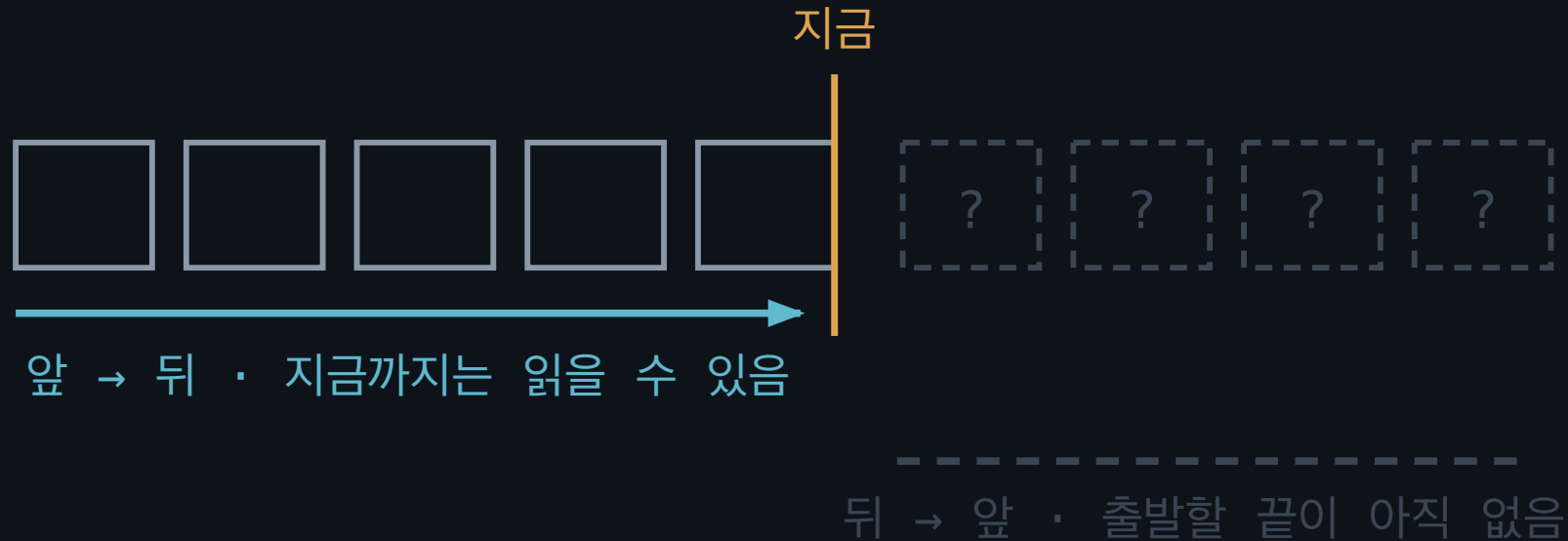
양방향 · 같은 줄을 앞뒤로 한 번씩 읽음

쓸 수 있는 조건

- 줄 전체가 이미 손에 있을 때만 씀
- 거꾸로 읽으려면 끝에서 출발해야 함
- 끝이 아직 없으면 출발을 못 함

HAR · 128칸이 다 찍힌 뒤에 창을 통째로 넣음

못 쓰는 자리



- 지금 막 들어오는 값을 보고 바로 판정해야 함
- 다음 칸이 뭘지 아직 모름 — 뒤가 없음
- 성능이 떨어지는 문제가 아니라 **쓸 수 없는 것임**

이 얘기를 맨 마지막에 꺼낸 이유

순서를 섞어도 답이 같은가

- 순서가 상관없으면 방향이라는 말이 성립하지 않음
- 양방향은 순서를 버리는 것이 아님
- 순서를 두 번 쓰는 것임

오늘 반복한 세 문장

표현만 바뀌가며 계속 나왔음

첫 문장

순서를 섞어도 답이 같은가
다르다면 길어도 매번 다른가

- 합으로 풀면 48.8%
- 채운 자리를 안 빼면 51%

두 번째 문장

다르다면 무엇을 넘길 것인가 사람이 정할까 · 학습시킬까

- 마지막 값 60.4% · 전부 넘기기는 길어지면 무너짐 · 두 비트 51.2%
- 학습에 맡기니 100% · 1,474개

세 번째 문장

넘긴 것이 끝까지 닿는가
못 닿는 게 기억 탓인가 학습 탓인가

- 학습하면 99칸도 닿음 · 맨 앞을 고칠 신호가 거의 안 와 느리게 배움
- HAR · RNN 82.5% · LSTM 90.6% · 원인은 안 잼

물은 것	그때 나온 것
순서를 섞어도 답이 같은가	합으로 풀면 48.8% 아침
무엇을 넘길 것인가	크기만 정하니 100% 오전
끝까지 닿는가	99스텝 기울기 10^{-28} 오후 · 학습 전 모델
문장마다 장면이 하나씩 붙어 있음	

그리고 오후에 하나가 더 붙었음

구조가 다르다고 성능이 다른 건 아님

그럼 무엇으로 고를 것인가 · LSTM 과 GRU 는 0.5%p 차이였음 · 답은 비용이었음

과제로 나가는 데이터



$[7352, 128, 9] \cdot [2947, 128, 9] - \text{개수} \cdot \text{길이} \cdot \text{칸마다 값}$

활동 여섯 가지를 맞이하는 문제임 · UCI HAR · CC BY 4.0

3 과 7 문제는 왜 오래 붙들었나

- 합으로는 원리적으로 못 푼다는 것을 알고 시작했음
- 그래서 50% 가 나왔을 때 뜻을 바로 말할 수 있었음
- 아는 문제로 먼저 연습한 것임

오늘 안 다룬 것

- Seq2Seq
- Attention
- Transformer

이름만 짚고 감 · 다음 과정에서 다룸

오늘 남길 것

자료가 아니라 묻는 순서

세 문장으로 묻고 아홉 줄로 내려감

오늘 여기까지